

Programmation orientée objet

Objectifs

- Comprendre la philosophie de l'approche orientée objet
- Apprendre à maîtriser ses principaux éléments
 - Classes, méthodes, encapsulation, héritage, polymorphisme...
- Apprendre à utiliser une démarche orientée objet pour
 - Structurer un programme
 - Résoudre un problème donné
- Apprendre à utiliser les classes existantes, pour ne pas réinventer la roue
 - Outils mathématiques, manipulation de chaînes de caractères...
 - Interfaces graphiques: fenêtres, boutons, boîtes de dialogue
- Comprendre les principes de la programmation événementielle
 - « Quand l'utilisateur clique ici, fais cela »

De manière générale

- Apprendre à écrire des programmes de qualité professionnelle
 - Conviviaux en environnement Windows
 - Qui ne « plantent » pas au premier événement inattendu
 - Lisibles et compréhensibles par des tiers

Pourquoi apprendre l'orienté objet ? (OO)

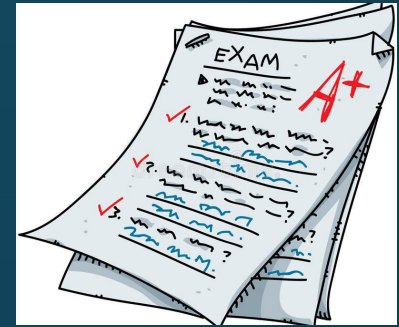
- Manière moderne de concevoir les programmes
 - Important de pouvoir se conformer à cette approche
- De nombreux langages sont fondamentalement orientés objets (Java, C#, ...)
- De nombreux outils (p.ex. tous les outils d'interfaçage graphique de C#) sont orientés objet
 - Compréhension des principes OO indispensable pour pouvoir les utiliser efficacement

Méthodologie

- Apprentissage par la pratique: nombreux exercices
 - /!\ La maîtrise des outils logiciels fait partie intégrante des capacités à acquérir
- Exercices de synthèse à réaliser hors séances
- Langage C#, mais les principes fondamentaux sont très proches du Java (et d'autres langages)
- Eviter que l'arbre ne cache la forêt
 - On va laisser de côté un certain nombre de concepts avancés
 - Classes abstraites, interfaces
 - On ne va pas voir toutes les classes, tous les outils...
- Idée: fournir de bonnes bases permettant une approche d'auto-apprentissage en fonction des besoins futurs



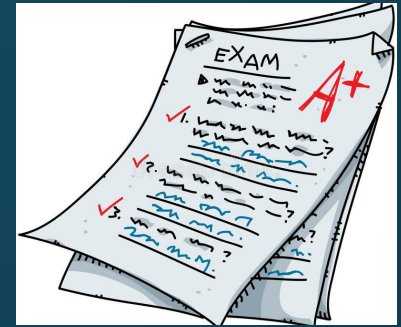
Critères de réussite et acquis d'apprentissage



L'étudiant sera capable [...] de concevoir, d'installer et d'utiliser des objets appropriés à la solution

- Pouvoir **définir, implémenter, compléter et interagir avec** des classes et des objets C# simples
- Pouvoir dériver de nouvelles classes par **héritage** d'une classe C# existante
- Pouvoir utiliser les classes du **framework C# et .Net** pour mettre en œuvre une **interface graphique** simple

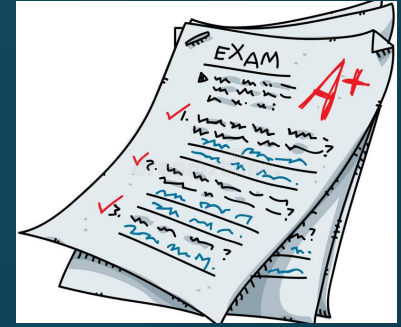
Autrement dit...



Pour réussir l'examen, vous devez pouvoir

- Définir des classes décrites en langage courant et créer des objets correspondants
- Faire interagir des classes entre elles
 - Classe composée d'autres classes
 - Classe faisant appel à d'autres classes
- Maîtriser la différence entre un attribut, une variable locale et un paramètre
- Construire une classe héritée d'une autre classe et l'utiliser
- Utiliser les classes .Net pour créer un projet (Windows Form) doté d'une interface graphique

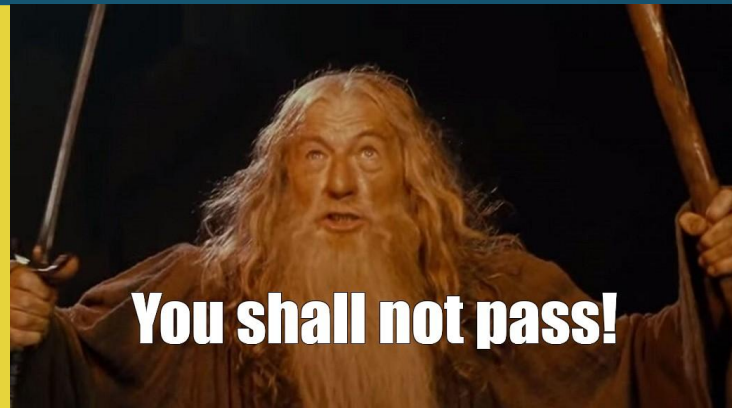
Autrement dit...



Pour réussir l'examen, vous devez pouvoir

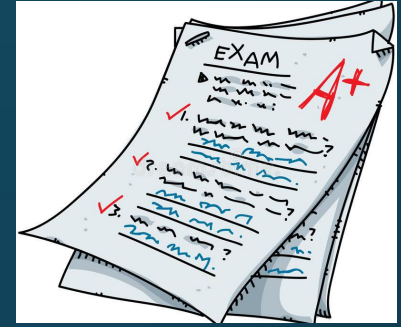
- Définir des classes décrites en langage courant et créer des objets correspondants
- Faire interagir des classes entre elles
 - Classe composée d'autres classes
 - Classe faisant appel à d'autres classes
- Maîtriser la différence entre un attribut, une variable locale et un paramètre

- Ces 3 critères sont fondamentaux:
- Si l'un des 3 n'est pas rempli...



iser

Et bien évidemment...



Vous devez être capable de transmettre votre projet !

- Le localiser sur le disque dur
- Créer une archive du projet complet
- La déposer sur Moodle

« Pour réussir l'examen, il faut remettre une réponse »

Expérimentation sur un exemple

Exemple introductif

- Récupérez l'application "transport"
- Idée:
 - Des **camions** transportent des **marchandises** d'un endroit à un autre
 - Les marchandises ont un poids, les camions ont une capacité limitée...
- Concentrez-vous sur la fonction Main() dans Program.cs
 - Comment créer un camion, transporter des marchandises ?
 - Essayez de transporter 300 kg de poires de Wavre à Gand
 - Testez les cas limite: que se passe-t-il si on surcharge un camion?
- Ne pas s'inquiéter des détails: voir l'idée générale
- Les textes qui s'affichent en vert sont produits « ailleurs » dans le code

Questions

- Qu'est-ce qui a (ou n'a pas) changé par rapport aux principes vus en première année?
 - Au niveau des variables
 - Au niveau des fonctions
- Pistes
 - Quelle est la valeur de la variable `CamionDeMax` en fin de programme?
 - Peut-on exécuter une ligne de code comme celle-ci?

```
RoulerVers("Bruxelles");
```
 - La fonction `DechargerTout()` n'affiche rien, n'a pas de paramètres ref/out et ne renvoie rien. Peut-on dire pour autant qu'elle ne fait rien ?

Principes généraux

Complexité du code

- Un programme = plusieurs milliers (millions) de lignes de code
- Important de le **structurer**
- Avantages
 - Organiser le raisonnement
 - Simplifier la maintenance
 - Permettre la réutilisation de composants
- Une première étape de structuration: l'approche procédurale
 - Via des fonctions, dont le rôle et l'interface sont bien définis
- Mais il y a moyen d'aller plus loin...

Complexité des données

- Les données que doit traiter un programme sont souvent complexes
 - Des personnes : nom, prénom, âge...
 - Des documents : contenu, auteur, date de création, droits d'accès...
 - Des fichiers multimédias : type, encodage, taille...
- On peut évidemment utiliser plusieurs variables

```
string NomPers1, NomPers2, PrénomPers1, PrénomPers2;  
int AgePers1, AgePers2;  
string[] Noms = new string[10];
```
- Mais ce n'est pas très confortable

Programmation OO : approche centrée sur les données

- Élément central: les **objets**
- Types complexes:
 - Rassemblant plusieurs **informations** (e.g. nom, âge, adresse...)
 - Egalement capables d'accomplir des **actions** (e.g. déménager, épouser...)
- Les données et les actions font intrinsèquement partie de l'objet
- Un objet peut lui-même être composé d'objets

Cuisinier

Nom

Prénom

Préparer()



Serveur

Nom

Prénom

ListeCommandes[...]

PrendreCommande()

Servir()



Client

Nom

Prénom

Argent

Payer()



Préparer(poulet)

Servir(poulet)

PrendreCommande(poulet)

Payer(poulet)

Approche « top-down » vs. « bottom-up »

- Approche procédurale : de haut en bas
 - Découper le problème à résoudre en tâches
 - Découper chaque tâche en sous-tâches
 - ... et ainsi de suite, jusqu'à obtenir des sous-sous-...-tâches simples à coder
- Approche orientée objet : de bas en haut
 - Définir des objets simples résolvant des éléments du problème
 - Les combiner en objets plus complexes
 - ... et ainsi de suite, jusqu'à la résolution du problème
- Tâcher de concevoir des objets génériques, qui pourront resservir dans d'autres contextes

Objets et classes

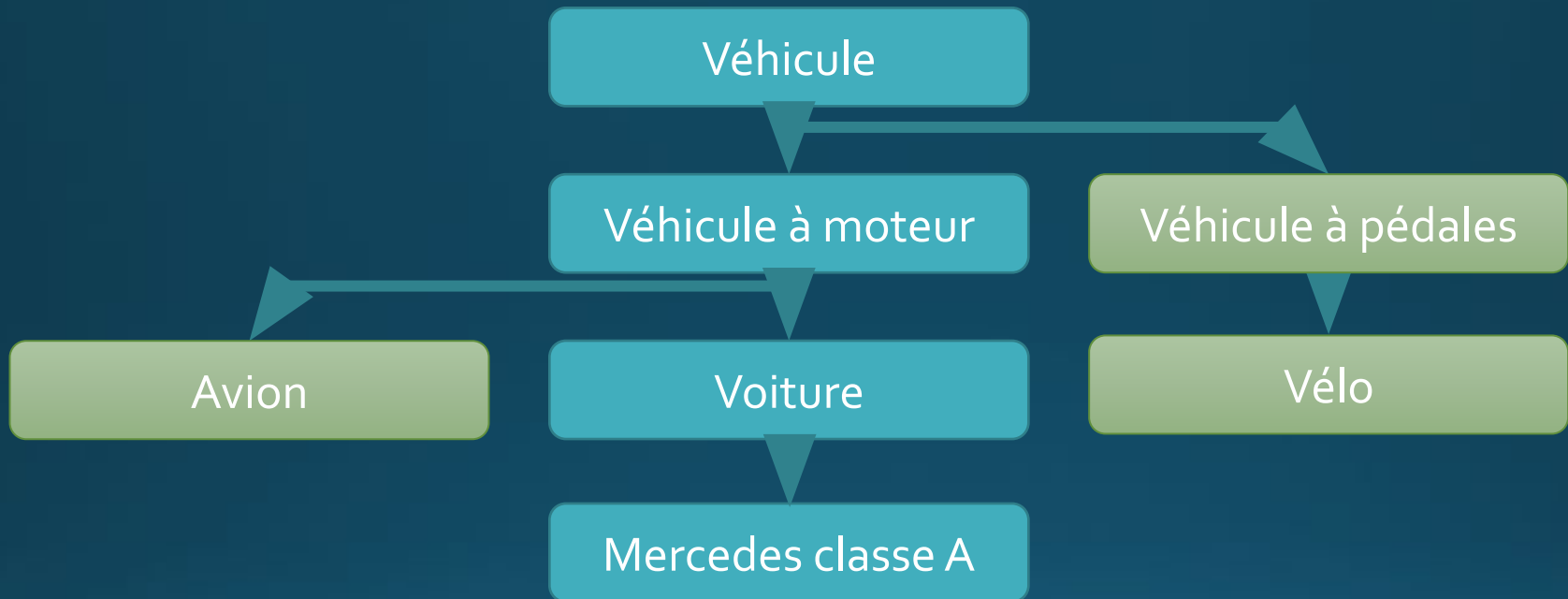
- Objets
- Un objet spécifique
- Classes
 - « Voiture »
 - « Opel Zafira »

La voiture de M. Martin



- Un objet appartient donc à une classe
 - (et même à plusieurs)

Hiérarchie de classes



- L'objet « voiture de M. Durand » appartient à toutes ces classes
- Autres classes (auxquelles il n'appartient pas)

Classe

- ≈ le « type » de l'objet
- Une **classe** regroupe
 - Des **attributs**, ou champs (≈ les variables en prog. procédurale)
 - Des **méthodes** (≈ les fonctions en prog. procédurale)
- Exemple

Personne

Nom

Prénom

Année de naissance

CalculAge()

Afficher()

En C#

... maintenant on définit
de nouvelles classes

/*! Jusqu'ici on écrivait
dans la classe Program...

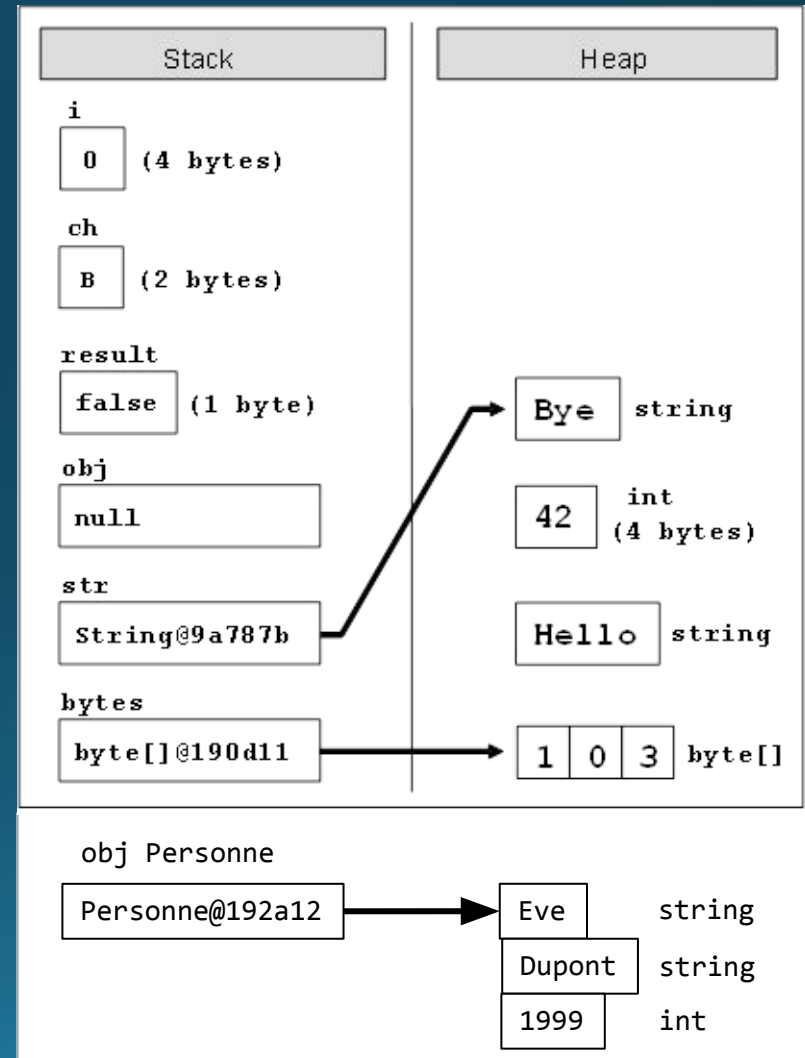
```
namespace Prog00
{
    class Personne
    {
        public string Nom;
        public string Prénom;
        public int AnnéeNaissance;
        public int Age;
        public void CalculAge(int Année)
        {
            Age = Année - AnnéeNaissance;
        }
        public void Afficher()
        {
            Console.WriteLine(Prénom + " " +
                               Nom + " (" + Age + " ans)");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            [...]
        }
    }
}
```

Objets

- La classe étant créée, il nous faut maintenant créer des objets appartenant à cette classe
 - Déclarer des variables de type `Personne`
`Personne Dupont;`
 - Créer des objets ...

Types composés et références

- Rappel
 - Les données composées sont stockées dans une autre zone de la mémoire
 - La variable contient une référence à cette zone
- Les classes sont des types composés
 - Une variable objet contient donc une référence vers la zone de stockage
 - L'objet doit être créé avant d'être affecté à une variable



Objets

- La classe étant créée, il nous faut maintenant créer des objets appartenant à cette classe
 - Déclarer des variables de type Personne
`Personne Dupont;`
 - Créer des objets de ce type et les affecter à ces variables
`Dupont = new Personne();`
 - Remarque: on dit que l'objet Dupont est une **instance** de la classe Personne
 - L'accès aux attributs et aux méthodes se fera ensuite via des appels comme
`Console.WriteLine(Dupont.Nom);`
`Dupont.CalculAge(2019);`

Attributs et méthodes

- **Attribut** : variable interne stockant une information propre à l'objet (e.g. nom, prénom, âge...)
- **Méthode** : fonction opérant sur un objet
 - Peut prendre des paramètres et/ou renvoyer des résultats
 - Peut avoir des variables locales
 - **Peut manipuler les attributs (les lire et/ou les modifier)**
 - Peut faire appel à d'autres méthodes
- Les méthodes font partie intégrante de la classe
- Idée : regrouper en une même entité tout ce qui concerne l'objet : **encapsulation**

La méthode agit sur l'objet

- Parfois (souvent), une méthode ne renvoie rien, mais se contente de modifier l'objet auquel elle appartient
- Exemple

```
class Personne
{
    [...]
    public int Age;
    public void CalculAge(int Année) {
        Age = Année - AnnéeNaissance;
    }
}
```

- Cette méthode calcule l'âge de la personne (en fonction de l'année actuelle), et met à jour l'attribut Age de l'objet
 - **Stockage « long terme »** : Age gardera sa valeur après la fin de la méthode
- Pour lire ensuite cet âge, on fera une autre requête à l'objet

Le point de vue à adopter



Le point de vue à adopter: celui des objets

- On **transmet** des informations à l'objet
- L'objet **se met à jour** sur base de certaines données
- On **demande** à l'objet de s'afficher
- ...
- Les communications avec un objet sont appelés des **messages**
(même si, en pratique, ce sont des appels de méthodes...)

Exemple d'utilisation de la classe Personne

```
Personne Prof = new Personne();  
Prof.Nom = "Koeune";  
Prof.Prénom = "François";  
Prof.AnnéeNaissance = 1973;  
Prof.CalculAge(2024);  
Prof.Afficher();
```

- *Remarque* : pas encore tout à fait conforme à la philosophie OO (accès direct aux attributs)
 - Nous y reviendrons

Remarque : appels « de dedans » vs. appels « de dehors »

- Quand on fait référence à un attribut ou à une méthode d'un objet depuis l'extérieur de la classe, il **faut** préciser l'objet et l'attribut/la méthode

```
Prof.Nom = "Koeune";  
Prof.Afficher();
```

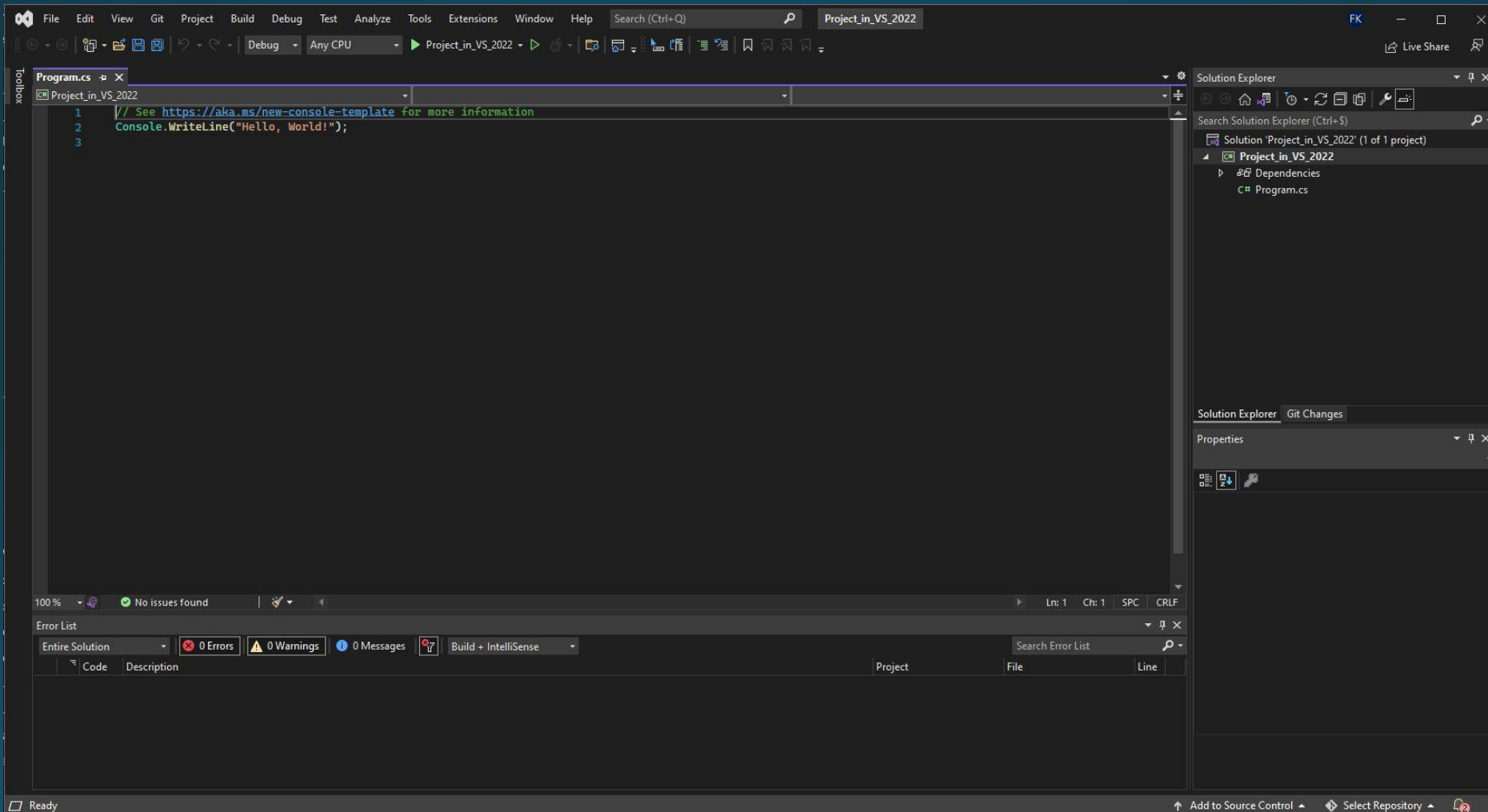
- Quand on se trouve à l'intérieur de la classe, on **ne peut pas** préciser l'objet
 - (= « Age et AnnéeNaissance de l'objet courant »)

```
class Personne {  
    public int AnnéeNaissance;  
    public int Age;  
    [...]  
    public void CalculAge(int Année) {  
        Age = Année - AnnéeNaissance;  
    }  
}
```

Exercice

- Une voiture se caractérise par une marque, un modèle, un numéro de série, un prix catalogue et un prix réel
 - Lors de la vente, le vendeur applique souvent un pourcentage de réduction sur le prix catalogue, ce qui donne le prix réel
- Créez une classe Voiture définissant
 - Ces différents attributs
 - Une méthode CalculPrixReel() prenant en entrée le taux de réduction
 - Une méthode Afficher() permettant à l'objet d'afficher ses données
- Créez ensuite plusieurs objets de classe Voiture pour tester votre classe, et exécutez des ventes avec taux de réduction
- Y a-t-il un conflit entre la méthode Afficher() de Voiture et celle de la classe Personne?

Avant cela...



Avant cela...

- Visual Studio 2022 (.NET 6) a changé
 - Fonction Main -> top-level statements
- Au lieu de déclarer une fonction Main, on peut écrire directement le code qu'elle doit exécuter
- Plus rapide, plus court
- En pratique, vous pouvez déclarer vos classes juste en-dessous

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");

1 reference
class Voiture
{
}
```

Exercice

- Une voiture se caractérise par une marque, un modèle, un numéro de série, un prix catalogue et un prix réel
 - Lors de la vente, le vendeur applique souvent un pourcentage de réduction sur le prix catalogue, ce qui donne le prix réel
- Créez une classe Voiture définissant
 - Ces différents attributs
 - Une méthode CalculPrixReel() prenant en entrée le taux de réduction
 - Une méthode Afficher() permettant à l'objet d'afficher ses données
- Créez ensuite plusieurs objets de classe Voiture pour tester votre classe, et exécutez des ventes avec taux de réduction
- Y a-t-il un conflit entre la méthode Afficher() de Voiture et celle de la classe Personne?

Méthodes de même nom

- Nous avons maintenant deux méthodes nommées Afficher()
 - L'une dans la classe Voiture
 - L'autre dans la classe Personne
- Ces deux méthodes coexistent sans problème
 - Aucune ambiguïté possible : la classe à laquelle appartient l'objet indique la méthode à appeler

```
Personne Prof = new Personne();  
Voiture AutoProf = new Voiture();  
[...]  
Prof.Afficher();      // méthode de la classe Personne  
AutoProf.Afficher(); // méthode de la classe Voiture
```

- C'est un exemple de **polymorphisme** (nous en verrons d'autres)

Constructeurs

Constructeur

- L'instruction new de la ligne

```
Prof = new Personne();
```

fait en fait plusieurs choses

- **Allouer** une zone mémoire pouvant contenir un objet Personne
 - **Initialiser** cet objet
 - **Renvoyer** son adresse pour être stockée dans Prof
- L'allocation de la mémoire et le renvoi de l'adresse sont pris en charge par le compilateur
 - Par contre, **l'initialisation de l'objet est de la responsabilité du concepteur de la classe**
 - Cela se fait via un **constructeur**

Constructeur

- Une méthode qui sera exécutée dès la création de l'objet
- Rôle: **initialiser l'état interne** de l'objet
 - Donner des valeurs aux champs
 - Ouvrir un fichier
 - Réserver des ressources (webcam...)
 - ...
- Un constructeur peut prendre des paramètres en entrée (valeurs initiales...)
- Plusieurs constructeurs peuvent être définis pour une même classe (nous y reviendrons)
- Remarque
 - En l'absence de constructeur, le compilateur en crée automatiquement un, qui ne fait rien
 - Mais il est de bonne pratique de définir des constructeurs dédiés

Constructeur : syntaxe

- Comme une méthode classique, mais
 - Doit porter le même nom que la classe
 - Généralement qualifié de public (nous y reviendrons)
 - Ne précise pas de type de retour, ni void

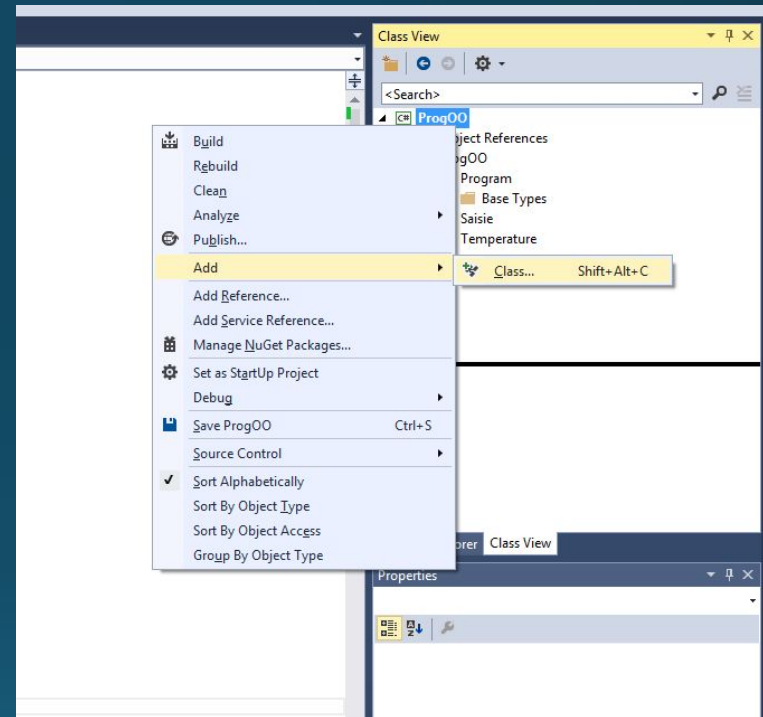
```
public Personne(string N, string P, int A) {  
    Nom = N;  
    Prénom = P;  
    AnnéeNaissance = A;  
}
```

- Utilisation du constructeur

```
Personne Prof = new Personne("Koeune", "François", 1973);
```


Remarque : classe dans un fichier séparé

- Un projet sous Visual Studio se compose de plusieurs fichiers
- Il est possible (et fréquent) de définir une classe dans un fichier séparé
- On peut le faire via le menu contextuel « Add Class »
- **Mais** il faut alors ajouter une directive `using NomDuProjet` au début de `Program.cs`



```
// See https://aka.ms/new-console-template for more information
using ProjetVoiture; ←

Console.WriteLine("Hello, World!");

Voiture AutoDeMax = new Voiture();
```

Mot réservé `this`

- Il arrive parfois que l'on doive explicitement faire référence à l'objet en cours
- C# définit pour cela le mot réservé `this`, signifiant « l'objet courant »
 - Cela permet (entre autres) d'utiliser le même nom pour une variable locale

```
class Personne
{
    public string Nom;
    public string Prenom;
    public int AnneeNaissance;
    public Personne(string Nom, string Prenom, int AnneeNaissance)
    {
        this.Nom = Nom;
        this.Prenom = Prenom;
        this.AnneeNaissance = AnneeNaissance;
    }
}
```

Exercice

- Définissez un constructeur pour votre classe Voiture, permettant de passer en paramètre les valeurs initiales
 - Que se passe-t-il maintenant si l'on appelle
`v = new Voiture();`

Exercice

- Un étudiant est caractérisé par
 - Un nom
 - 4 notes
 - La moyenne de ces notes
 - Un résultat d'ensemble qui vaut « Réussite » ou « Echec »
- Définissez la classe Etudiant en conséquence et dotez-la d'un constructeur et de méthodes permettant à un objet Etudiant de calculer sa moyenne et son résultat et de s'afficher

Exercice

- Définissez une classe Dé modélisant un dé à 6 faces
- Elle devra au moins offrir un constructeur et les méthodes
 - Lancer() : le dé « tombe » sur une nouvelle face entre 1 et 6
 - Lire() : « regarde » sur quelle face le dé est, sans le relancer
- Remarque:
 - Cette classe présente un défaut si nous voulons l'utiliser avec plusieurs dés
 - En effet, plusieurs dés initialisés simultanément utiliseront des générateurs aléatoires identiques, et produiront donc toujours le même résultat
 - Nous verrons une solution avec les attributs et méthodes statiques

Remarque

- Que fait votre programme dans le cas suivant ?

```
static void Main(string[] args)
{
    De De1 = new De();
    Console.WriteLine("J'ai tiré un : " + De1.Lire());
}
```

- Il ne peut pas afficher 0 !
 - Zéro n'est pas une face admissible sur un dé
 - **Responsabilité du concepteur de l'objet** : s'assurer que l'objet est à tout moment dans un état autorisé

```
class Dé {  
    public int Valeur;  
    public Random Rng;  
    public Dé() {  
        Rng = new Random();  
        Valeur = 1;  
    }  
    public void Lancer() {  
        Valeur = Rng.Next(6);  
        Valeur = Valeur + 1;  
    }  
    public int Lire() {  
        return Valeur;  
    }  
}
```

Encapsulation

Interface vs. implémentation

- Un principe fondamental en programmation moderne
Bien distinguer **ce que** le code fait de **comment** il le fait
- Exemples
 - `Console.ReadLine()` lit une valeur tapée au clavier
 - Pas besoin de savoir comment C# communique avec le clavier
 - `rng.Next(i)` renvoie un nombre aléatoire de $[0, i[$
 - Pas besoin de savoir comment C# fait pour produire ce nombre
- Déjà mis en œuvre en programmation procédurale
- Poussé encore plus loin en programmation OO
 - Encapsulation** : on peut décider de rendre les attributs/méthodes/classes accessibles depuis l'extérieur ou non

Encapsulation

Méthodes et attributs publics



Méthodes et attributs privés



Encapsulation en C#

- Les attributs et méthodes peuvent être qualifiées de
 - **public** : possible d'accéder à cet attribut ou à cette méthode, même en dehors de la classe
 - **private** : accès possible uniquement pour les membres de la classe
 - **protected** : accès possible seulement pour les membres de la classe et des classes dérivées (héritage : nous y reviendrons)
 - **internal** : accès possible seulement depuis le même « bloc de code »
- De même, une classe peut être qualifiée de
 - **public** : on peut créer partout un objet de cette classe
 - **internal** : on ne peut créer d'objet de cette classe que depuis le même « bloc de code »

Exemple

- Une classe `Fichier` pourrait contenir
 - Des attributs, auxquels l'utilisateur de la classe a accès
 - Nom du fichier, taille...
 - Des attributs `private` pour la « cuisine interne »
 - Numéros de secteur du fichier sur le disque dur, position courante dans le fichier...
 - Des méthodes `public`
 - Ouvrir, Lire, Fermer...
 - Des méthodes `private`
 - Gestion de conflits entre processus, copies entre disque et RAM...

Public ou private ?

- La question à se poser
 - Y a-t-il besoin de pouvoir accéder à cet élément pour utiliser la classe ?
- Le bon réflexe (surtout au début)
 - « Par défaut, c'est private »

Reprenons notre classe Dé

```
class Dé {  
    private int Valeur;  
    private Random Rng;  
    public Dé() {  
        Rng = new Random();  
        Valeur = 1;  
    }  
    public void Lancer() {  
        Valeur = Rng.Next(6);  
        Valeur = Valeur + 1;  
    }  
    public int Lire() {  
        return Valeur;  
    }  
}
```

Pas besoin de savoir qu'elle stocke sa valeur courante dans un int

Pas besoin de savoir qu'elle utilise un générateur aléatoire

Par contre, l'utilisateur doit pouvoir créer un objet

Ainsi que lancer un dé

Et lire le résultat

Avantages

- L'utilisateur de la classe
 - N'est pas distrait par des éléments dont il n'a pas besoin
 - Ne risque pas de « faire de bêtises »
 - Par exemple `d1.Valeur = 7;`
- Si plus tard le concepteur décide de modifier l'implémentation de la classe

(par exemple d'utiliser un générateur aléatoire sécurisé)

 - Il sait que cette modification n'a **aucun risque** de poser de problèmes à un quelconque programme utilisant la classe, puisqu'il est **impossible** que ces programmes aient utilisé les éléments `private`
 - Tant qu'il conserve l'interface (les éléments publics et le comportement de la classe), aucun souci à craindre

Méthode privée

- Une méthode peut aussi être qualifiée de privée
- Elle ne peut alors être appelée que de l'intérieur de la classe
- Utile pour organiser la "cuisine interne"

Exercice

- Créez une classe température, qui permettra de stocker une température et de l'exprimer aussi bien en degrés Celsius que Fahrenheit
- Plus précisément, cette classe devra
 - Posséder un constructeur prenant en entrée la température et l'unité
 - Posséder 2 méthodes EnCel() et EnFahr() pour renvoyer la température dans l'unité désirée
 - Posséder une méthode Chauffage(), déterminant s'il faut allumer le chauffage (on allume le chauffage en dessous de 15°C)
 - ... et tous les éléments qui vous sembleront nécessaires
- Bien réfléchir aux
 - Signatures des méthodes (prend quoi, renvoie quoi?)
 - Qualification public ou private
- Rappel: °C $\square$ °F : $y = x * 9/5 + 32$ (10°C $\square$ 50°F)
 °F $\square$ °C : $y = (x - 32) * 5/9$ (23°F $\square$ -5°C)

Pour vérifier si votre classe tient la route...

Les opérations ci-dessous (une ligne de code par opération) doivent générer des réponses cohérentes:

- Créer un objet température à 10°C
- Afficher cette température en °C
- L'afficher en °F
- L'afficher à nouveau en °F
- L'afficher en °C

```

class Temperature {
    private double TempInCelsius;
public Temperature(double T, char Unité) {
    if (Unité == 'C') TempInCelsius = T;
    else TempInCelsius = (T - 32) * 5.0 / 9;
    // Pour simplifier: par défaut, c'est Fahrenheit
}
public double EnCel() {
    return TempInCelsius;
}
public double EnFar() {
    return TempInCelsius * 9.0 / 5 + 32;
}
public bool Chauffage() {
    if (TempInCelsius < 15.0) return true;
    else return false;
}
}

```

```
class Program
{
    static void Main(string[] args)
    {
        Temperature T = new Temperature(54, 'F');
        Console.Write("Il fait " + T.EnCel() + "°C, c'est-à-dire ");
        Console.WriteLine(T.EnFar() + "°F");
        if (T.Chauffage()) Console.WriteLine("Allumez le
chauffage");
        else Console.WriteLine("Il fait bien assez chaud comme ça");
    }
}
```

Variante

```
class Temperaturev2
{
    private double TempInCelsius;
    private double TempInFahrenheit;
    public Temperaturev2(double T, char Unité) {
        if (Unité == 'C') {
            TempInCelsius = T;
            TempInFahrenheit = T * 9.0 / 5 + 32;
        } else {
            TempInFahrenheit = T;
            TempInCelsius = (T - 32) * 5.0 / 9;
        }
    }
    public double EnCel() {
        return TempInCelsius;
    }
    public double EnFar() {
        return TempInFahrenheit;
    }
    public bool Chauffage() {
        if (TempInCelsius < 15.0) return true;
        else return false;
    }
}
```

Variante

- La version 2
 - Est implémentée différemment
 - Stocke (via des attributs) les deux expressions °C et °F
 - Un peu plus de travail à l'initialisation (constructeur)
 - Un peu moins à l'affichage (EnCel(), EnFahr())
 - Mais offre la même interface
 - Peu importe la « cuisine interne » : les deux variantes offrent les mêmes fonctionnalités et s'utilisent de la même manière
- Quelle est la bonne version ?
 - Les deux sont très bien
- *Remarque:*
 - En pratique, pas besoin de donner des noms différents à ces variantes : l'utilisateur n'a pas à se soucier de quelle version il utilise

Un point important à garder en tête

- Avec la version 2, il y a a priori un risque que l'objet se retrouve dans un état incohérent
 - P.ex. `TemplnCelsius` = 3 et `TemplnFahrenheit` = 5
- Mais pas de problème : **la classe est conçue de manière telle que ceci ne puisse jamais arriver**
 - Aucun bloc de code *de la classe* ne peut modifier un attribut sans modifier également l'autre de manière cohérente
 - Ces attributs ne sont pas accessibles *hors de la classe*
- Représenter la température de cette manière est une bonne pratique, du moment qu'on a assuré ces garde-fous
 - Ce serait différent si les attributs étaient publics

Accès aux attributs:

Getters & Setters

Accès aux attributs: *Getters & Setters*

- Une pratique assez courante en programmation OO : **cacher tous les attributs**
 - Tous les attributs sont private
 - L'accès aux attributs (quand il est nécessaire !) se fait via des méthodes
 - **GetX()** pour **lire** l'attribut X
 - **SetX()** pour **écrire** l'attribut X
- Un peu plus lourd à coder, mais plus en phase avec la philosophie OO
 - (et il existe des raccourcis, à découvrir par vous-mêmes 😊)

Getters et Setters : exemple

```
class Personne {
    private string Nom;
    private string Prenom;
    public string GetNom() {
        return Nom;
    }
    public string GetPrenom() {
        return Prénom;
    }
    public void SetNom(string s) {
        Nom = s;
    }
    public void SetPrenom(string s) {
        Prénom = s;
    }
    [...] }
class Program {
    static void Main(string[] args) {
        [...]
        p1.SetPrenom("Joseph");
        string s = p1.GetNom();
    }
}
```

Getters et Setters : à quoi ça sert ?

- Revenons à la variante 2 de Température, en supposant les attributs publics
- Supposons qu'un utilisateur fasse directement
`T.TempInCelsius = 25;`
- L'objet est maintenant dans un état incohérent !
 - Les deux expressions de la température ne sont plus équivalentes
- Un *Setter* permet de s'assurer que toute modification est appliquée correctement à l'ensemble de l'objet

```
class Tempv2 {  
    public double TempInCelsius;  
    public double TempInFahrenheit;  
    public Tempv2(double T, char Unité) {  
        if (Unité == 'C') {  
            TempInCelsius = T;  
            TempInFahrenheit = T * 9 / 5 + 32;  
        } else {  
            TempInFahrenheit = T;  
            TempInCelsius = (T - 32) * 5 / 9;  
        }  
    }  
    public double EnCel() {  
        return TempInCelsius;  
    }  
    [...]
```

Exercice

- Complétez la classe *Température* avec un ou des *Setters*
 - (pas besoin de *Getter* : les méthodes *InCel()* et *InFar()* jouent déjà ce rôle)
- Au passage
 - Il y a maintenant besoin à plusieurs endroits de faire des conversions de °C vers °F et vice-versa
 - Profitez-en pour définir des méthodes permettant de convertir un réel de l'un vers l'autre
 - Ces méthodes doivent-elles être privées ou publiques?

Exercice

- Une sonde spatiale transporte un équipement de pointe ne pouvant fonctionner à des températures extrêmes. Lors de son activation, la sonde teste la température de l'équipement : si elle est inférieure à -50°C , un système de chauffage de conception américaine est activé. Ce système augmente la température de 25°F . Un nouveau test de température est alors effectué, qui active à nouveau le système de chauffage si nécessaire, et ainsi de suite jusqu'à atteindre une température acceptable.
- Lors de l'activation initiale, la température est de -271°C . Quelle sera la température de fonctionnement de l'équipement ?

Exercice: chiffrement de César

- Pour protéger ses secret militaires, Jules César utilisait un code secret
 - Chaque lettre du message était remplacée par celle qui se trouve 3 positions avant dans l'alphabet (D $\rightarrow$ A, E $\rightarrow$ B, ..., A $\rightarrow$ X)
 - Bien évidemment, on peut généraliser la méthode avec un autre nombre que 3
- Ecrire une classe César permettant de chiffrer et déchiffrer des messages selon ce système
 - Le constructeur prend en paramètre la clef (càd la valeur du décalage)
 - La classe offre 2 méthodes Chiffrer() et Déchiffrer() prenant une chaîne de caractères et renvoyant la chaîne chiffrée/déchiffrée
- Pour simplifier, on suppose que le message est composé uniquement de lettres minuscules non accentuées
- Indices :
 - Pour connaître la longueur d'une variable s de type string : s.Length
 - Pour lire le ième caractère de s : s[i]
 - Pour remplacer une lettre par celle qui la suit dans l'alphabet, on peut simplement lui ajouter 1 (mais attention aux dernières lettres de l'alphabet)

Surcharge de méthodes

Surcharge de méthodes

- Il est parfois utile de disposer de plusieurs variantes de la même méthode
- En C#, plusieurs méthodes d'une même classe peuvent avoir le même nom, du moment que
 - Le **nombre**
 - Et/ou le **type** des paramètres est différent
- Intuition : pas d'ambiguïté au moment de l'appel
- C'est un autre exemple de **polymorphisme**
- Nous l'avons d'ailleurs largement utilisé avec `Console.WriteLine()`

Exemple

```
class Personne {  
    public void Afficher() {  
        Console.WriteLine(Prenom + " " + Nom + " (" + Age + " ans)");  
    }  
    public void Afficher(string Msg) {  
        Console.Write(Msg + " ");  
        Console.WriteLine(Prenom + " " + Nom + " (" + Age + " ans)");  
    }  
    [...]  
  
    /* Appel des méthodes */  
    p1.Afficher();  
    p1.Afficher("Les données sont: ");
```

Surcharge de méthodes : limites

- Les méthodes de même nom ne peuvent coexister que si elle se distinguent par leurs paramètres
- Une distinction sur le type de valeur renvoyée ne suffit pas
 - Par exemple, ces 4 méthodes ne peuvent avoir le même nom
`public int machin(int a, int b);`
`public double machin(int a, int b);`
`public void machin(int a, int b);`
`private void machin(int a, int b);`
- Le nom des paramètres ne suffit pas non plus
`public int machin(int a, int b);`
`public int machin(int x, int y);`

Cas particulier: surcharge du constructeur

- Le constructeur est une méthode (presque) comme une autre
 - Il peut lui aussi être surchargé
 - Très utile pour prévoir plusieurs moyens de créer un objet, en donnant les valeurs initiales de différentes façons

Exemple

```
class Temperature {  
    private double TempInCelsius;  
    public Temperature()  
    {  
        TempInCelsius = 0;  
    }  
    public Temperature(double T, char Unité)  
    {  
        if (Unité == 'C') TempInCelsius = T;  
        else TempInCelsius = (T - 32) * 5 / 9;  
    }  
    public Temperature(Temperature T)  
    {  
        TempInCelsius = T.EnCel();  
    }  
}
```

Un constructeur sans valeur initiale

Un constructeur qui initialise à partir de données « brutes »

Un constructeur qui copie un objet existant

Saisie des valeurs au sein du constructeur

- Une possibilité intéressante est que le constructeur saisisse immédiatement les valeurs tapées au clavier
- Exemple

```
public Temperature() {  
    string s;  
    Console.Write("Introduisez la température (en °C): ");  
    s = Console.ReadLine();  
    TempInCelsius = Double.Parse(s);  
}
```

- Remarque : ne peut pas coexister avec l'un des constructeurs définis ci-dessus (pourquoi?)

Exercice

- Complétez votre classe Température avec
 - Un constructeur comme ci-dessus, mais permettant de saisir au clavier la température *et* l'unité
- Testez-la en créant quelques objets de cette classe

Remarque: surcharge d'opérateurs

- C# permet aussi de redéfinir des **opérateurs** (e.g. +, -, *, ...)
- Il est donc possible de définir une nouvelle classe et des opérateurs sur cette classe, et d'écrire ensuite des instructions comme

```
ClasseMachin m1, m2, m3;  
[...]  
m1 = m2 + m3;
```

- Apprendre à le faire sort du cadre de ce cours, mais nous utiliserons des classes ainsi surchargées, telles que
 - String : "abc" + "xyz", "ABC" + 3, ...
 - DateTime : permet de calculer la distance entre deux dates, etc.

Exercice

- Créez une classe CompteBancaire
 - Un compte bancaire se caractérise par un numéro de compte, un solde et un découvert autorisé
 - La classe offrira en outre des méthodes permettant de verser et retirer (si le solde est suffisant) de l'argent

ToString()

- Une méthode générique produisant un résumé du contenu d'un objet sous la forme d'une chaîne de caractères
 - Appelée dans divers cas où l'objet doit être affiché (p.ex. `Console.WriteLine(<objet>)`)
- Une version "par défaut" existe (par héritage)
 - Souvent utile de le surcharger avec un traitement plus spécifique

```
class Personne
{
    [...]
    public override string ToString()
    {
        return Nom + " " + Prenom;
    }
}
```

Lié à l'héritage (nous y reviendrons)

Attributs, méthodes et Classes statiques

Attributs et méthodes statiques

- Les méthodes et attributs sont normalement liés à un objet de la classe
 - Exemple: si p1 et p2 sont des instances de la classe Personne
 - p1.Nom et p2.Nom ont des valeurs différentes
 - p1.Affiche() et p2.Affiche() ne font pas la même chose
- Il arrive cependant qu'on veuille attacher un attribut ou une méthode à la classe elle-même, et non à un de ses objets
 - Exemple: imaginons une classe Cercle, permettant entre autres de calculer l'aire d'un cercle
 - Nous aurons besoin de la valeur de Pi, mais celle-ci est toujours la même, quel que soit le cercle considéré

(En fait, Pi est directement accessible via la classe Math, mais peu importe)

```
class Cercle {
```

```
    static public double Pi = 3.1416;
```

Attribut statique

```
    private double Rayon;
```

```
    public Cercle(double Rayon) {
```

```
        this.Rayon = Rayon;
```

```
    }
```

```
    public double Diamètre() {
```

```
        return 2 * Pi * Rayon;
```

Accès depuis l'intérieur
de la classe

```
    }
```

```
    public double Aire() {
```

```
        return Pi * Rayon * Rayon;
```

```
    }
```

```
}
```

```
[...]
```

```
Cercle c;
```

```
c = new Cercle(1);
```

```
Console.WriteLine("Diamètre: " + c.Diamètre() + ", aire: " + c.Aire());
```

```
Console.WriteLine("Pour mémoire, Pi = " + Cercle.Pi);
```

Accès depuis l'extérieur
de la classe

Attribut statique

- Existe indépendamment de tout objet de la classe
 - (même si aucun objet n'a été créé)
- Est partagé par tous les objets de la classe
- Accessible
 - De l'intérieur de la classe, comme d'habitude
 - De l'extérieur, avec la notation `NomClasse.NomAttribut`
 - P.ex. `Cercle.Pi` (utiliser le nom de l'objet, p.ex. `c1.Pi`, serait une erreur)

Attribut statique: autre exemple

- Reprenons la classe Dé
- Un de ses défauts est lié à la façon dont fonctionne le générateur aléatoire de C#
 - Ce générateur
 - Possède un état interne
 - Qui est initialisé à la création (par défaut, à partir de l'heure exacte)
 - Et évolue (de manière déterministe) à chaque utilisation
 - Mais donc
 - Si 2 générateurs sont créés au même moment, ils produiront exactement la même séquence de nombres
 - Et, par conséquent, idem pour 2 Dés
- Mais les différents objets "Dé" ont-ils besoin chacun de leur propre RNG ?

```
class Dé {  
    private int Valeur;  
    static private Random Rng = new Random();  
    public Dé() {  
        Valeur = 1;  
    }  
    public void Lancer() {  
        Valeur = Rng.Next(6);  
        Valeur = Valeur + 1;  
    }  
    public int Lire() {  
        return Valeur;  
    }  
}
```

RNG comme attribut statique

Avantages:

- Moins de ressources consommées
- Initialisation unique pour toute la classe

/!\ Attribut statique $\neq$ constante

- Un attribut statique n'est pas forcément constant
 - Attaché à la classe
 - N'existe qu'en un seul exemplaire
 - Mais peut évoluer
- Exemple: un compteur indiquant le nombre d'objets créés


```
class ClasseAvecCompteur {  
    static private int NbObjets = 0;  
    private int Contenu;  
    public ClasseAvecCompteur(int c) {  
        Contenu = c;  
        NbObjets++;  
    }  
    public int GetContenu() { return Contenu; }  
    public static int GetNbObjets() {  
        return NbObjets;  
    }  
}  
  
static void Main(string[] args) {  
    ClasseAvecCompteur o1, o2, o3;  
    o1 = new ClasseAvecCompteur(17);  
    o2 = new ClasseAvecCompteur(4);  
    Console.WriteLine("o2 contient: " + o2.GetContenu());  
    Console.WriteLine("Vous avez créé " +  
        ClasseAvecCompteur.GetNbObjets() + " objets de cette classe");  
}
```

Méthode statique

- Existe indépendamment de tout objet de la classe
 - (même si aucun objet n'a été créé)
- Est partagée par tous les objets de la classe
- Accessible
 - De l'intérieur de la classe, comme d'habitude
 - De l'extérieur, avec la notation `NomClasse.NomMéthode()`
- */!* Une méthode statique ne peut pas faire appel à des attributs non statiques, ni à des méthodes non statiques
 - (Logique, quand on y réfléchit)
- *Remarque* : toutes les fonctions que nous avons écrit l'an dernier étaient statiques (pourquoi ?)

Classes statiques

- Une classe peut également être qualifiée de static
 - Tous ses attributs et méthodes sont static
 - Aucun objet de cette classe ne peut être créé
- Exemple: la classe Console
 - Rassemble tous les outils d'accès input/output
 - Pas d'objet de cette classe (il n'y a que LA console)
 - Accès aux méthodes comme décrit ci-dessus:
 - `Console.WriteLine("Hello");`
 - `S = Console.ReadLine();`
- Autre exemple: la classe Math

Une classe statique (vraiment) utile: Saisie

- Il est souvent nécessaire de
 - Afficher une question à la console (e.g. « Entrez un entier »)
 - Lire la réponse sous forme de string
 - Convertir vers le type désiré (e.g. int)
- Nous allons définir une classe Saisie permettant d'automatiser ce processus
 - Les appels prendront une forme comme
`int i = Saisie.LireInt("Veuillez entrer votre âge");`
 - (et idem pour lire un réel, un booléen, ...)
 - Pas besoin de créer d'objets « Saisie » □ classe statique

Une classe statique (vraiment) utile: Saisie

- La méthode LireString()
 - Prend en paramètre le message à afficher
 - Renvoie la chaîne lue au clavier

```
static class Saisie {  
    public static String LireString(String msg)  
    {  
        Console.Write(msg);  
        return Console.ReadLine();  
    }  
}
```

- Remarques
 - La classe *et* la méthode sont toutes deux qualifiées de static
 - Impossible de déclarer des objets de classe Saisie (et pas de constructeur)

Une classe statique (vraiment) utile: Saisie

- Exercice: complétez la classe Saisie
 - Enrichissez-la de méthodes pour pouvoir lire des entiers, des réels et des booléens
 - N'oubliez pas que ces méthodes peuvent faire appel à LireString, qui fait déjà la moitié du boulot !
 - Pas de gestion des erreurs à ce stade

Programmes
utilisant
plusieurs classes

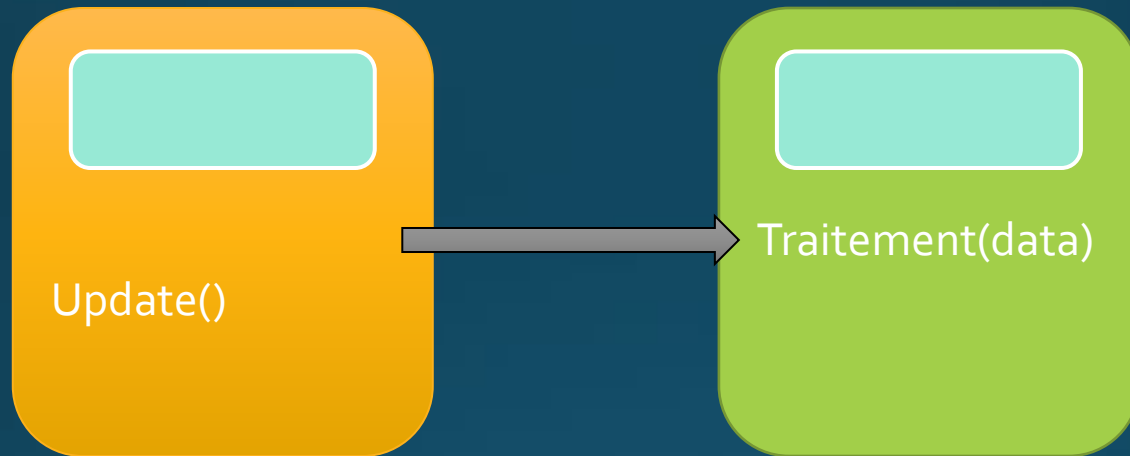
Programmes utilisant plusieurs classes

- Jusqu'ici, nos programmes utilisaient une seule classe
 - (On a parfois défini plusieurs classes dans le programme, mais sans interactions entre elles)
- Généralement, un « vrai » programme
 - Définit plusieurs classes qui implémentent différents éléments du problème à résoudre
 - Ces classes, et leurs objets, interagissent et collaborent pour résoudre le problème
 - Mais cela doit se faire en respectant la philosophie OO
 - Les interactions se font uniquement via les interfaces prévues (encapsulation)
 - Chaque objet est responsable de son état, et communique avec les autres via des messages

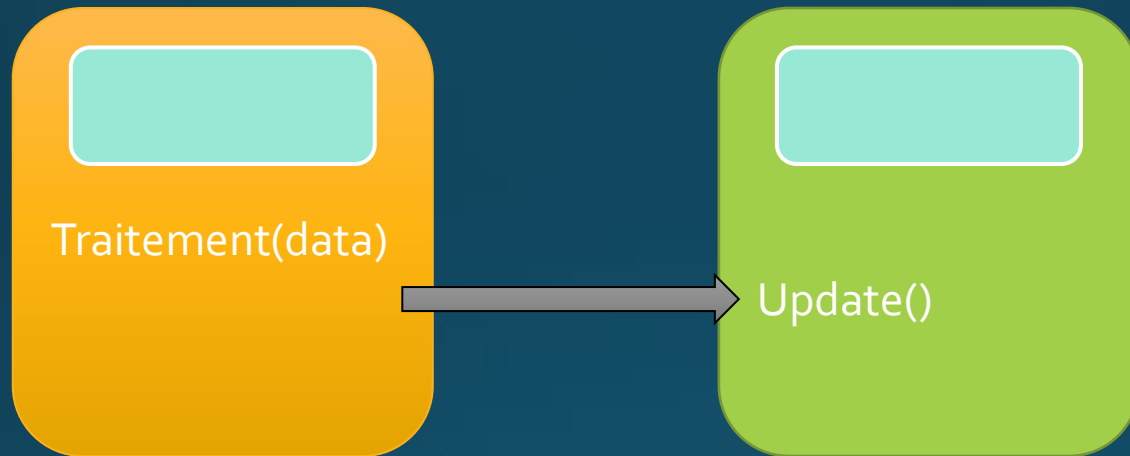
Cas de figure typiques (1/4)



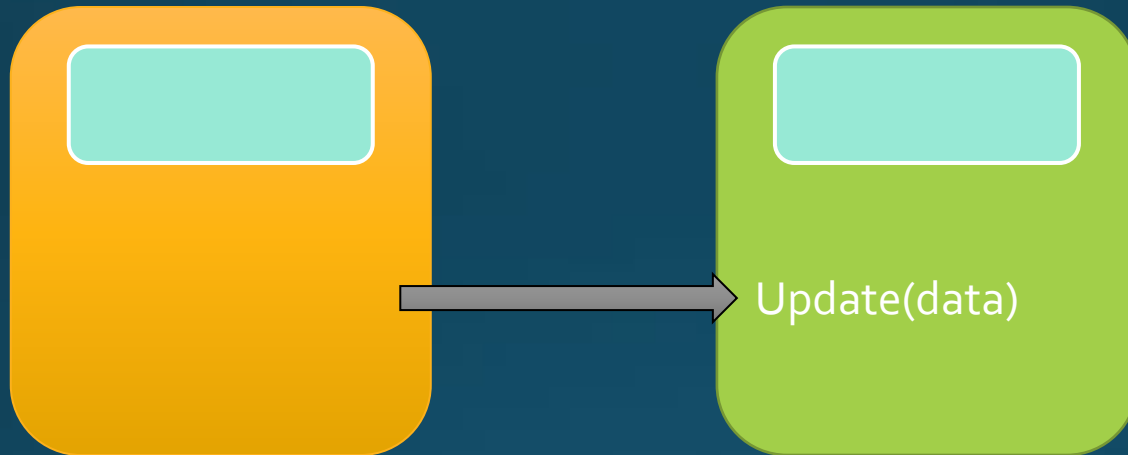
Cas de figure typiques (2/4)



Cas de figure typiques (3/4)



Cas de figure typiques (4/4)



Bien évidemment...

- ... c'est très simplifié !
- Souvent
 - Plusieurs méthodes doivent être appelées
 - Les appels s'enchaînent en cascade
 - La mise à jour de O1 demande d'abord une mise à jour de O2
 - Qui elle-même nécessite qu'O2 demande l'aide de O3
 - Etc.
- Mais ces schémas-type de base se retrouvent régulièrement

Exemple: paiement entre 2 personnes (version 1)

- Contexte: plusieurs personnes possèdent de l'argent et effectuent des paiements entre elles
- Option 1: deux classes
 - Personne, possédant
 - 2 attributs: Nom, Argent
 - Les méthodes classiques GetXX(), Afficher(), ...
 - Banque, possédant
 - Une méthode Payer()

```
class Personne {  
    private string Nom;  
    private int Argent;  
    public Personne(string Nom, int Capital) {  
        this.Nom = Nom;  
        this.Argent = Capital;  
    }  
    public string GetNom() {  
        return Nom;  
    }  
    public int GetArgent() {  
        return Argent;  
    }  
    public void SetArgent(int NouvelleValeur) {  
        Argent = NouvelleValeur;  
    }  
    public override string ToString() {  
        return Nom + ": " + Argent + " euros";  
    }  
}
```

```
class Banque {  
    public void Payer(Personne p1, Personne p2, int montant) {  
        p1.SetArgent(p1.GetArgent() - montant);  
        p2.SetArgent(p2.GetArgent() + montant);  
    }  
}  
  
class Program {  
    static void Main(string[] args) {  
        Personne Dupont = new Personne("Dupont", 2500);  
        Personne Durand = new Personne("Durand", 2000);  
        Banque CreditFacile = new Banque();  
        CreditFacile.Payer(Dupont, Durand, 150);  
        Console.WriteLine(Dupont.ToString());  
        Console.WriteLine(Durand.ToString());  
    }  
}
```


Observations

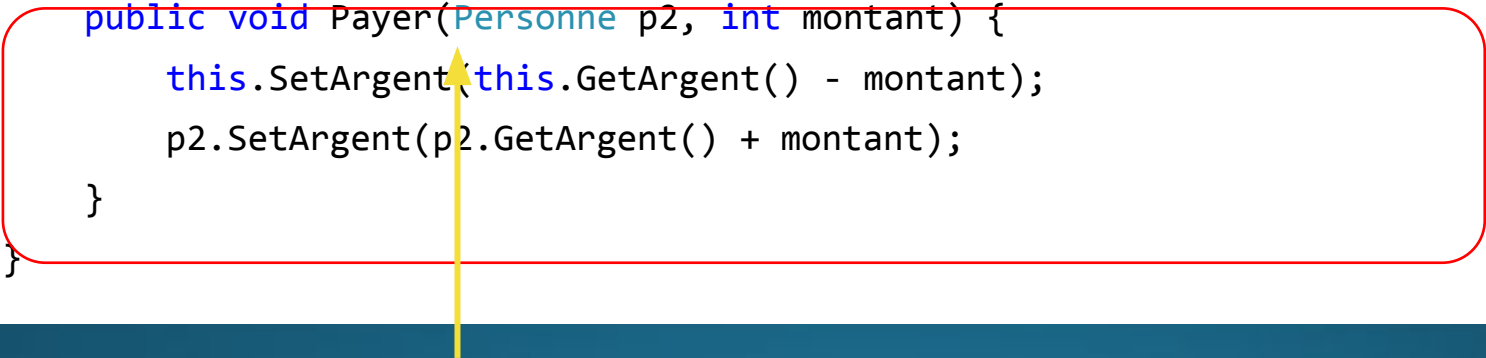
```
class Banque {  
    public void Payer(Personne p1, Personne p2, int montant) {  
        p1.SetArgent(p1.GetArgent() - montant);  
        p2.SetArgent(p2.GetArgent() + montant);  
    }  
}
```

- Payer() a besoin
 - De lire l'argent disponible chez chaque Personne
 - De modifier ces valeurs pour effectuer le paiement
- Mais elle ne peut pas manipuler ces valeurs directement !
 - A la place, elle demande aux objets Personne
 - De retourner les valeurs nécessaires
 - De se mettre à jour en fonction des nouvelles valeurs
 - Ces demandes s'appellent des **envois de messages**

Exemple: paiement entre 2 personnes (version 2)

- Dans le cas présent, la classe Banque ne sert pas à grand-chose
- Option 2 : une seule classe
 - Personne, possédant
 - 2 attributs: Nom, Argent
 - Les méthodes classiques GetXX(), Afficher(), ...
 - Une méthode Payer()

```
class Personne {
    private string Nom;
    private int Argent;
    public Personne(string Nom, int Salaire) {
        this.Nom = Nom;
        this.Argent = Salaire;
    }
    public string GetNom() { return Nom; }
    public int GetArgent() { return Argent; }
    public void SetArgent(int NouvelleValeur) {Argent=NouvelleValeur;}
    public override string ToString() {
        return Nom + ": " + Argent + " euros";
    }
    public void Payer(Personne p2, int montant) {
        this.SetArgent(this.GetArgent() - montant);
        p2.SetArgent(p2.GetArgent() + montant);
    }
}
```



Test: où est passé p1 ?

```
static void Main(string[] args)
{
    Personne Dupont = new Personne("Dupont", 2500);
    Personne Durand = new Personne("Durand", 2000);
    Dupont.Payer(Durand, 150);
    Console.WriteLine(Dupont.ToString());
    Console.WriteLine(Durand.ToString());
}
```

- Payer() ne prend plus que 2 paramètres
 - Les identités des intervenants ne sont pas passées de manière symétrique
 - L'un (Dupont) est l'objet appelant
 - L'autre (Durand) est passé en paramètre
- Cas de figure très courant en prog. OO

Observations

```
public void Payer(Personne p2, int montant) {  
    this.SetArgent(this.GetArgent() - montant);  
    p2.SetArgent(p2.GetArgent() + montant);  
}
```

- Il n'y a plus d'objet Banque
- Mais les accès se font toujours via envoi de messages
 - On demande à p2 les valeurs nécessaires
 - On lui demande de se mettre à jour avec les nouvelles valeurs
- En fait, dans le cas présent, on pourrait s'en passer...

Exemple: paiement entre 2 personnes (version 3)

- **private** permet l'accès aux seuls membres de la classe
 - Mais p1 et p2 sont tous deux des Personne
 - p1 aussi est membre de la classe !
 - Il peut donc manipuler directement les attributs, même private

```
public void Payerv3(Personne p2, int montant)
{
    this.Argent = this.Argent - montant;
    p2.Argent = p2.Argent + montant;
}
```

- Les objets peuvent directement manipuler les attributs d'autres objets de la même classe
 - Même s'ils sont private
- Permet un code plus compact
- Ne respecte pas le principe de l'envoi de messages
 - (mais le programmeur est censé maîtriser les détails d'implémentation de « sa » classe)

Exercice

- Programmer un mini-jeu de rôle
 - Un héros possède un certain nombre de points de vie
 - Il peut attaquer un autre héros
 - Il lance un dé à 6 faces
 - Le résultat est retranché des points de vie de l'adversaire
 - Un duel se déroule comme suit
 - Chaque combattant attaque à tour de rôle
 - Le duel s'arrête à la mort d'un des combattants
- Programmez-le
 - D'abord en suivant le principe de l'option 1 ci-dessus, avec une classe Arène qui gère les combats
 - Puis selon l'option 2, en n'utilisant que la classe Héros

Association: composition et agrégation

Association: composition et agrégation

- Nous avons vu que différents objets pouvaient interagir entre eux
- On peut également rencontrer des associations « long terme » entre objets
- 2 formes d'association:
 - Un objet peut être formé à partir d'autres objets : **composition**
 - Des objets peuvent être connectés entre eux : **agrégation**

Composition

- Un objet possède des attributs
- Ces attributs peuvent eux-mêmes être des objets
 - Qui possèdent à leur tour des méthodes et des attributs
 - ... et ainsi de suite
- Exemple : une Voiture possède un Moteur

```
class Moteur
{
    private string Type;
    private string Puissance;
    public Moteur(string Type, string Puissance)
    {
        this.Type = Type;
        this.Puissance = Puissance;
    }
    public void Afficher()
    {
        Console.WriteLine("type: " + Type + " puissance: " + Puissance);
    }
}
```

```
class Voiture
```

2 constructeurs sont prévus

```
{
```

```
    private string Marque;
```

```
    private Moteur Mot;
```

```
    public Voiture(string Marque, string TypeMoteur, string PuissanceMot)
```

```
    {
```

```
        this.Marque = Marque;
```

```
        Mot = new Moteur(TypeMoteur, PuissanceMot);
```

```
    }
```

L'un qui créera un objet Moteur

```
    public Voiture(string Marque, Moteur Mot)
```

```
    {
```

```
        this.Marque = Marque;
```

```
        this.Mot = Mot;
```

```
    }
```

L'autre qui utilisera un Moteur existant

```
    public void Afficher()
```

```
    {
```

```
        Console.Write("Marque: " + Marque + " Moteur ");
```

```
        Mot.Afficher();
```

```
    }
```

```
}
```

Encapsulation : l'affichage du moteur se fait par un envoi de message à l'objet correspondant

```
static void Main(string[] args)
{
    Moteur mot1 = new Moteur("Diesel", "95");
    Voiture Twingo = new Voiture("Twingo", "Essence", "55");
    Voiture Astra = new Voiture("Astra", mot1);
    Twingo.Afficher();
    Astra.Afficher();
}
```

Agrégation

- Des objets peuvent également être connectés sans que l'un soit « contenu » dans l'autre
 - Une personne peut être mariée à une autre personne
 - Une personne peut être employée par une entreprise
 - ...
- Composition vs. agrégation
 - Pas de différence point de vue code (attributs d'un objet)
 - Mais représentent des réalités différentes
 - « Fait partie de » (composition) vs. « a un » (agrégation)
 - Un composant n'existe pas seul
 - Pour plus d'infos:
 - <https://javarevisited.blogspot.com/2014/02/iffERENCE-between-association-vs-composition-vs-aggregation.html>
 - <https://algodaily.com/lessons/association-aggregation-composition-casting>

Remarque: mot réservé `null`

- Il arrive qu'une variable de type composé n'ait pas de valeur
 - Un tableau qui n'a pas été réservé (avant l'appel `new`)
 - Un objet qui n'a pas été créé
 - ...
- Ceci est indiqué par le mot réservé `null`
 - On ne peut pas accéder aux éléments d'un tel objet (il n'y en a pas!)
 - Mais on peut tester si l'objet existe

```
if (p1 == null)
    {...}
```
 - Et « vider » un objet existant

```
p1 = null;
```


/!\ Une erreur très fréquente

- Dès le moment où un objet *peut* en contenir un autre (mais n'en contient pas forcément un)
 - Il *faut* vérifier si c'est le cas ou non
 - Sinon le programme plante
 - (on ne peut pas appeler les méthodes d'un objet null !)

```
Console.WriteLine(Jean.GetVoiture().GetMoteur()); // Erreur !
```

```
if (Jean.GetVoiture() != null)  
    Console.WriteLine(Jean.GetVoiture().GetMoteur());
```

Types « nullables »

- Une donnée de type référence peut donc prendre la valeur null
- En fait, C# permet aussi de le faire avec les types valeur
 - Indiqué en ajoutant un '?' après le type

```
int age;           // On a toujours un âge
int? nombrePrefere; // On peut ne pas avoir de nombre préféré

age = 17;           // Age = null serait une erreur
nombrePrefere= null;
```

□ Parfois utile, mais nécessitera aussi des tests de consistance!

Nullable reference type

- C# 8 a introduit la notion de *nullable reference type*
 - Permet d'annoter une variable de type référence en précisant si une valeur null est admissible ou non
 - (Lié à la notion de required/optional field en bases de données)
 - Comme pour le type valeur, indiqué par un '?' après le type

```
private Adresse AdressePrincipale;  
private Adresse? AdresseSecondaire;
```

- Si cette option est activée^(*), le compilateur générera un warning dès qu'une donnée non nullable risque de prendre une valeur nulle
 - Cf. warning que vous avez déjà rencontré (variable string...)

```
5 static string Machin(bool condition)  
6 {  
7  if (condition) return "Hello";  
8  else return null;  
9 }
```



“Possible null reference return”

Nullable reference type

- C# 8 a introduit la notion de *nullable reference type*
 - Permet d'annoter une variable de type référence en précisant si une valeur null est admissible ou non
 - (Lié à la notion de required/optional field en bases de données)
 - Comme pour le type valeur, indiqué par un '?' après le type

```
private Adresse AdressePrincipale;  
private Adresse? AdresseSecondaire;
```

- Si cette option est activée^(*), le compilateur générera un warning dès qu'une donnée non nullable risque de prendre une valeur nulle
 - Cf. warning que vous avez déjà rencontré (variable string...)
 - Demande de la discipline
 - Mais bonne pratique, très utile pour détecter « l'erreur très fréquente » ci-dessus

(*) Par défaut depuis VS 2022

```

internal class Personne
{
    private string Nom;           // On a toujours un nom
    private string Prenom;
    private string? Surnom;      // Mais pas forcément un surnom
    private Adresse AdressePrincipale;
    private Adresse? AdresseSecondaire;

    public Personne(string Nom, string Prenom, Adresse Adresse)
    {
        this.Nom = Nom;
        this.Prenom = Prenom;
        AdressePrincipale = Adresse;
    }
    public string? GetSurnom() // Elle pourrait renvoyer null
    {
        return this.Surnom;
    }
}

```

... et de nombreux autres aspects !

- Test explicite de valeur null
- “*null-forgiving operator* ‘!’
- ...



The Billion Dollar Mistake might be prevented with C# 8



Mathias

5 min read



Listen

We've all tried the first time and fine it it wasn't

I call it my billion-dollar mistake. It was the invention of the null reference in 1965. At that time, I was designing the first comprehensive type system for references in an object oriented language (ALGOL W). My goal was to ensure that all use of references should be absolutely safe, with checking performed automatically by the compiler. But I couldn't resist the temptation to put in a null reference, simply because it was so easy to implement. This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years.

Source: QCon 2019

This blog post will explore how "null" can become more useful and less annoying with some new features in C# 8.

<https://medium.com/@mathiaslykkegaardlorenzen/the-billion-dollar-mistake-might-be-prevented-with-c-8-24b134c5ab7b>

Exercice

- Examinez le programme « NullableTypes » sur Moodle pour vous familiariser avec quelques autres éléments liés aux types « nullable »

Exercice

- Revenons à notre mini-jeu de rôle
 - Une arme se caractérise par
 - Un nom
 - Un bonus, qui sera ajouté au résultat du dé
 - Créez la classe Arme correspondante
 - Modifiez la classe Héros pour que chaque héros possède une arme et s'en serve lors de ses attaques
 - Que faut-il modifier dans la méthode Duel() ?

Exercice

- Modifiez la classe Personne pour qu'une personne puisse posséder une voiture
 - Des méthodes permettront également à une personne d'acheter et de vendre une voiture
 - Remarque : une personne peut ne pas posséder de voiture
 - Prévoyez également une méthode ToString renvoyant une description de la personne et de sa voiture
- Modifiez la classe Personne pour qu'une personne puisse avoir un conjoint
 - Créez une classe Commune permettant de
 - Marier 2 personnes
 - Prononcer un divorce

Tableaux & listes d'objets

Tableaux, listes... d'objets

- On peut bien évidemment créer des listes, des tableaux... d'objets
- Il ne faut simplement pas oublier d'exécuter deux niveaux de « new »
 - Un pour créer la structure
 - L'autre pour créer les objets de cette structure

```
Temperature[] thermometres;  
thermometres = new Temperature[10];  
  
for (int i = 0; i < 10; i++)  
{  
    thermometres[i] = new Temperature(i * 10, 'C');  
}  
  
for (int i = 0; i < thermometres.Length; i++)  
{  
    Console.WriteLine(thermometres[i].EnFahr() + "°F");  
}
```

Contient null

Contient un tableau de 10 objets null

... qu'il faut créer individuellement

Exercice

- Complétez la classe `Personne` pour qu'une personne puisse avoir des enfants
- Prévoyez entre autres des méthodes
 - `AjouterEnfant()`
 - `AfficherEnfants()`

Héritage

Exemple introductif

- Récupérez le projet “Intro-Heritage” sur Moodle
- Ce projet met en oeuvre 2 classes
 - Personne
 - Une personne possédant un nom, une somme d'argent... et pouvant effectuer diverses actions (dépenser, se marier...)
 - Rien que du très classique à ce niveau
 - Employé
 - Un employé, pouvant faire ce que fait une personne, plus d'autres actions
 - Classe héritée de personne afin d'en étendre les fonctionnalités
- Observez comment la classe Employe
 - Réutilise des fonctionnalités existantes
 - Ajoute de nouvelles fonctionnalités
 - Modifie certaines fonctionnalités

Objectif

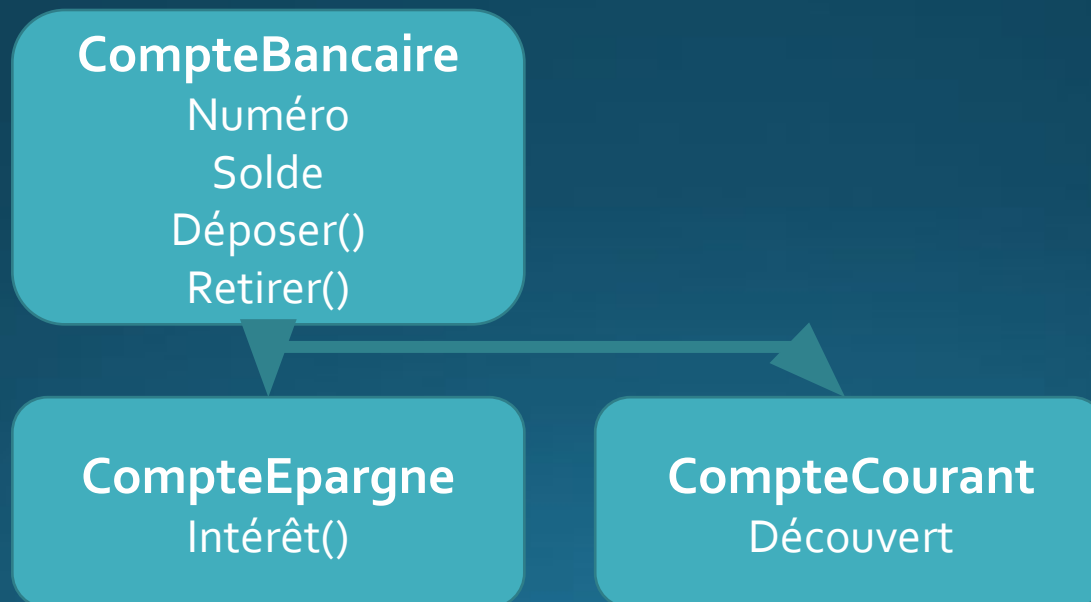
- Il arrive souvent que l'on veuille créer des classes plus spécialisées à partir d'une classe de base
 - Un Employé
 - Est une Personne : Nom, DateNaissance...
 - Possédant des attributs et méthodes supplémentaires : Salaire, Fonction, ...
 - Un Avion
 - Est un Véhicule : NombrePlaces, Embarquer(), ...
 - Possédant en outre : TypeMoteur, AppelRadio(), ...
 - Un AvionDeChasse
 - Est un Avion, possédant en outre : Armement, Attaquer(), ...
- L'héritage permet de réutiliser et de spécialiser une classe existante

Principe

- Une **classe de base** (ou **classe mère**)
- Est **étendue** pour créer une **classe dérivée**
(ou **classe fille**, ou **sous-classe**)
- La classe étendue possède
 - Les attributs et méthodes de la classe de base
(sauf si elle les écrase)
 - Des attributs et méthodes propres
- Remarque
 - La classe fille peut à son tour être étendue (« petite-fille »)
 - Et ainsi de suite

Exemple

- Considérons la classe CompteBancaire
 - Tout compte bancaire a un numéro et un solde
 - Seuls les comptes courants autorisent un découvert
 - Seuls les comptes épargne rapportent des intérêts



```

class CompteBancaire {
    private double Solde;
    private long Numéro;
    public CompteBancaire(long Numéro) { this.Numéro = Numéro; Solde = 0; }
    public double GetSolde() { return Solde; }
    public void SetSolde(double Solde) { this.Solde = Solde; }
    public void Déposer(double Montant) { Solde += Montant; }
    public bool Retirer(double Montant) {
        if (Montant > Solde) {
            Console.WriteLine("Solde insuffisant");
            return false; }
        else {
            Solde -= Montant;
            return true; }
    }
    public void Afficher() {
        Console.WriteLine("Numéro: " + Numéro + ", solde: " + Solde);
    }
}

```

```
class CompteEpargne : CompteBancaire
```

```
{
```

```
    private double TauxIntérêt;
```

```
    public CompteEpargne(long Numéro, double Taux) : base(Numéro)
```

```
    {
```

```
        TauxIntérêt = Taux;
```

```
    }
```

```
    public void Intérêts()
```

```
    {
```

```
        SetSolde(GetSolde() * (1 + TauxIntérêt));
```

```
    }
```

```
}
```

La classe hérite de la classe CompteBancaire

Et des méthodes et attributs propres

Elle possède donc toutes les méthodes et attributs
de la classe de base (sauf les private)

Constructeur

```
public CompteEpargne(long Numéro, double Taux) : base(Numéro)
{
    TauxIntérêt = Taux;
}
```

- Le constructeur opère en 2 temps
 - D'abord il appelle le constructeur de la classe de base
 - Ensuite il exécute son code propre
- Remarque
 - Un constructeur de la classe de base est toujours appelé
 - Si la séquence « : base(...) » n'est pas mentionnée, le compilateur appelle automatiquement le constructeur par défaut de la classe de base
 - Pour autant, bien sûr, qu'il n'ait pas été écrasé

Public, private et protected

- L'encapsulation est toujours appliquée
 - Des attributs et méthodes private ne sont pas accessibles en dehors de la classe
 - ... même depuis une classe fille
 - CompteCourant ne peut par exemple pas accéder directement à Solde
- Mais on peut le permettre avec le qualificatif **protected**
 - Accessible depuis la classe ET ses sous-classes
 - Mais toujours pas de l'extérieur

```
class CompteBancaire {
    protected double Solde;
    protected long Numéro;
    [...]
}

class CompteEpargne : CompteBancaire
{
    [...]
    public void Intérêts()
    {
        Solde = Solde * (1 + TauxIntérêt);
//        SetSolde(GetSolde() * (1 + TauxIntérêt));
    }
}
```

Exercice

- Définissez une classe `Personne`, possédant
 - Des attributs `Nom`, `Prénom`
 - Un constructeur et une méthode `Afficher()`
 - (Vous pouvez réutiliser la classe `Personne` définie précédemment)
- Définissez ensuite une sous-classe `Belge`, possédant
 - Un attribut supplémentaire `Communaute` (`Wallon`, `Flamand`)
 - Une méthode `GetCommunaute()`

Redéfinition de méthodes et d'attributs

- Certaines méthodes de la classe dérivée doivent se comporter différemment de celles de la classe de base
 - Quand on affiche un objet de type Belge, on voudrait voir sa communauté
 - Un compte courant autorise généralement un découvert
 - Un retrait depuis un compte courant ne se passe donc pas comme un retrait classique (le solde peut devenir négatif, tant qu'il reste au-dessus du découvert)
- Il faudrait donc
 - Adapter la méthode dans la classe dérivée pour qu'elle se comporte différemment de celle de la classe de base
 - Mais si possible sans devoir introduire de nouvelle méthode, portant un nouveau nom

□ L'héritage permet de le faire

Redéfinition de méthodes et d'attributs

- En prog. OO, il est assez courant « d'écraser » des méthodes de la classe de base
 - Nouvelle méthode, possédant son code propre
 - Mais portant le même nom et la même signature que celle de la classe de base
 - La nouvelle méthode masque donc celle de la classe de base
- Il y a 2 manières de le faire
 - La redéfinition (new)
 - Les méthodes virtuelles (virtual/override)

```
class CompteCourant : CompteBancaire
```

```
{
```

```
    public double DécouvertMax;
```

```
    public CompteCourant(long Numéro, double Découvert): base(Numéro) {  
        DécouvertMax = -Découvert;
```

```
    }
```

Redéfinition de la méthode Retirer() de CompteBancaire

```
    public bool Retirer(double Montant) {  
        if ((Solde - Montant) < DécouvertMax) {  
            Console.WriteLine("Solde insuffisant");  
            return false;
```

```
        }
```

```
        else {
```

```
            Solde -= Montant;
```

```
            return true;
```

```
        }
```

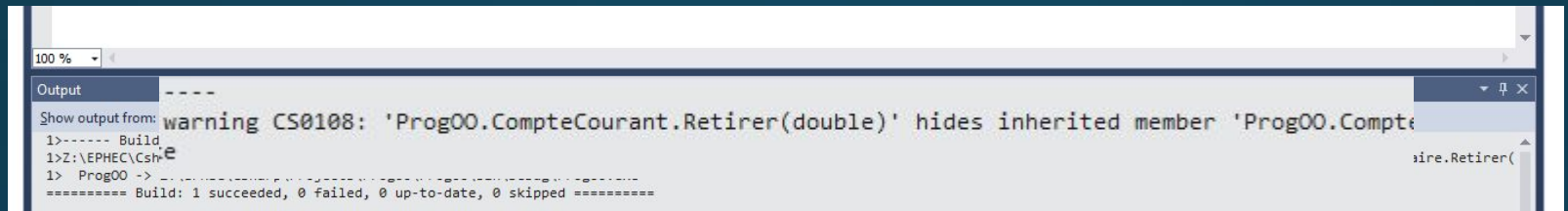
```
    }
```

```
}
```

//!\ Va générer
un warning

Redéfinition et new

- Le code ci-dessus va générer un warning à la compilation



The screenshot shows the 'Output' window in Visual Studio. The 'Show output from:' dropdown is set to 'Build'. The output text is as follows:

```
-----  
warning CS0108: 'Prog00.CompteCourant.Retirer(double)' hides inherited member 'Prog00.Compte  
1>----- Build  
1>Z:\EPHEC\Csh  
1> Prog00 ->  
-----  
Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped -----
```

- Garde-fou introduit par C# pour éviter une redéfinition accidentelle
- Pour bien montrer que c'est intentionnel, qualifier la méthode de **new**

```

class CompteCourant : CompteBancaire
{
    double DécouvertMax;
    public CompteCourant(long Numéro, double Découvert): base(Numéro) {
        DécouvertMax = -Découvert;
    }
    public new bool Retirer(double Montant) {
        if ((Solde - Montant) < DécouvertMax) {
            Console.WriteLine("Solde insuffisant");
            return false;
        }
        else {
            Solde -= Montant;
            return true;
        }
    }
}

```

Exercice

- Redéfinissez la méthode Afficher dans la classe Belge pour qu'elle affiche également la communauté

Héritage et type(s) d'objet

- Un objet d'une classe dérivée est aussi un objet de sa (ses) classe(s) de base
 - Un compte épargne est un compte bancaire
 - Un Belge est une personne
- On peut donc rencontrer des instructions comme

```
Personne p3 = new Belge("De Wever", "Bart", "Flamand");
```

- Cela a des conséquences importantes lors de la redéfinition de méthodes
- `/!\` Le contraire n'est pas permis

```
Belge p3 = new Personne(); // Erreur !
```

Exercice

- Exécutez le code suivant

```
Personne p1 = new Personne("Hollande", "François");  
Belge p2 = new Belge("Michel", "Charles", "Wallon");  
Personne p3 = new Belge("De Wever", "Bart", "Flamand");  
p1.Afficher();  
p2.Afficher();  
p3.Afficher();
```

- Que se passe-t-il ?
 - Et, surtout, pourquoi ?


```
Personne p1 = new Personne("Hollande","François");  
Belge p2 = new Belge("Michel", "Charles", "Wallon");  
Personne p3 = new Belge("De Wever", "Bart","Flamand");  
p1.Afficher();  
p2.Afficher();  
p3.Afficher();
```

Que se passe-t-il et pourquoi ?

- L'objet p1 est de type Personne
 - p1.Afficher() utilise donc la méthode de la classe Personne
- L'objet p2 est de type Belge
 - p2.Afficher() utilise donc la méthode de la classe Belge
- L'objet p3 est de type Personne !
 - Nous y avons mis un objet de type Belge
 - (ce qui est permis, puisque Belge est une sous-classe de Personne)
 - Mais ça, le compilateur ne le « sait » pas
 - Et, en général, ne peut pas le savoir, puisque l'assignation est dynamique
 - p3.Afficher() utilise donc la méthode de la classe Personne

```
Personne p1 = new Personne("Hollande","François");  
Belge p2 = new Belge("Michel", "Charles", "Wallon");  
Personne p3 = new Belge("De Wever", "Bart","Flamand");  
p1.Afficher();  
p2.Afficher();  
p3.Afficher();
```

- Cet exemple-ci est un peu extrême
 - Pourquoi avoir donné le type Personne à p3 ???
- Mais, dans la pratique, il arrive très souvent qu'un objet prenne ainsi le type d'une de ses classes-mères
 - Nous en verrons un exemple dans un prochain exercice
- C# offre un moyen de forcer le programme à vérifier le « véritable » type de l'objet durant l'exécution : les **méthodes virtuelles**

Méthodes virtuelles

- La méthode dans la classe de base doit être qualifiée de `virtual`
 - = « peut éventuellement être écrasée »
- La méthode dans la classe dérivée doit être qualifiée de `override`
 - = « écrase une méthode virtuelle »
- Remarque
 - Techniquement, ceci s'appelle de la *liaison dynamique*

```
class Personne
{
    [...]
    public virtual void Afficher()
    {
        Console.WriteLine(Nom + " " + Prénom);
    }
}
class Belge : Personne
{
    [...]
    public override void Afficher()
    {
        Console.WriteLine(Nom + " " + Prénom + " (" + Communauté + ")");
    }
}
```

new vs. virtual/override : résumé

new

- Le type « réel » de l'objet n'est pas pris en compte
 - « Si un Belge est stocké dans une variable Personne, il est considéré comme une Personne »
- Risque de ne pas exécuter la méthode attendue si ce genre de situation peut se produire
- Plus rapide

virtual/override

- Le type « réel » de l'objet est vérifié au moment de l'exécution
- La méthode la plus spécifique est toujours choisie
- Mais cela ralentit l'exécution

Exercice

- Reprenez la classe Héros, et définissez une sous-classe mage
 - Le mage possède un attribut entier Puissance
 - Lorsqu'il attaque, le mage n'utilise pas d'arme. A la place, il lance autant de dés à 6 faces que sa puissance et retranche le total des dés du nombre de points de vie de son adversaire
- Que se passe-t-il (et pourquoi) si l'on masque la méthode
 - Avec new ?
 - Avec virtual/override ?
- Quelle est la bonne solution ?

Accès aux attributs et méthodes: hiérarchie des classes

- Lors de l'appel d'une méthode, le « système » cherche une méthode portant ce nom
 - D'abord dans la classe de l'objet
 - Puis dans la classe-mère
 - Et ainsi de suite, jusqu'à rencontrer la première méthode adéquate
- Idem pour les attributs
- Remarques
 - La signature de la méthode (nombre et types de paramètres) est évidemment prise en compte dans la recherche
 - Les méthodes private des classes-mères ne sont pas considérées, puisqu'elles ne sont pas accessibles aux classes dérivées

Accès à une méthode de la classe de base

- Il peut arriver qu'une méthode désire accéder à une méthode de la classe de base
- Exemple
 - La classe dérivée redéfinit Afficher() pour afficher les attributs supplémentaires
 - Mais voudrait pouvoir utiliser Afficher() de la classe mère pour afficher les premiers attributs
- Cela peut se faire via le mot réservé **base**

```
public override void Afficher() {  
    base.Afficher();  
    Console.WriteLine("-->" + Communauté);  
}
```

- Remarque : idem pour les attributs

Type exact d'un objet: **is**

- Il est parfois utile de déterminer le type exact d'un objet
 - Cela peut se faire via le mot réservé **is**

```
public void Duel(Héros p2)
{
    if (p2 is Mage)
        Console.WriteLine("Contre un mage ?!? " + Nom + " prend des risques");
    [...]
```

Tentative de transtypage: **as**

- Le mot réservé `as` permet de tenter une conversion de type
 - Si l'objet visé est bien du type désiré, il est renvoyé sous la forme d'un objet de ce type
 - Sinon, `null` est renvoyé
- Exemple

```
Héros Robb = new Héros("Robb", 50, null);  
Mage Gandalf = new Mage("Gandalf", 25, 3);  
Mage m = Gandalf as Mage;    // m reçoit l'objet Gandalf  
Mage m2 = Robb as Mage;      // m2 reçoit null
```

Qualificatif sealed

- Il est possible d'interdire de dériver des sous-classes à partir d'une classe donnée
- Cela se fait via le qualificatif **sealed**

```
sealed class Mage : Héros  
{  
    [...]
```

- Semble peu utilisé en pratique
- Mais de nombreuses classes de Microsoft sont sealed

Non discuté dans ce cours

- Divers concepts de l'héritage dépassent le cadre de ce cours introductif
 - Classes abstraites
 - Interfaces
 - Méthodes d'extension
 - ...
- Mais certains de ces concepts seront introduits dans le cours « Prog OO : Java »
 - Et sont largement similaires en C#

Les classes String, StringBuilder et DateTime¹⁶⁵

Objectifs

- Découvrir l'intérêt des classes String, StringBuilder, DateTime et Math
- De manière générale, découvrir l'intérêt d'utiliser des classes existantes pour de nombreux traitements courants
 - Ne pas réinventer la roue
- Mais le découvrir par auto-apprentissage
 - Apprendre à lire la doc (MSDN...) pour comprendre le fonctionnement d'une classe

Principe

Pour chacun des exercices qui suivent

- Relativement peu de code à écrire
- Les classes visées offrent des méthodes qui effectuent une large part du travail
- Il s'agit donc
 - D'identifier les méthodes adéquates
 - De comprendre comment elles fonctionnent
 - Quels arguments demandent-elles ?
 - Que renvoient-elles ?
 - Quels objets, quels attributs... contiennent-ils les éléments de réponse ?

Comment **ne pas** faire l'exercice

- Demander au voisin d'expliquer comment il a fait
- Chercher sur Google « C sharp mettre string majuscules »
... ou demander à ChatGPT
 - Ira très vite : code à copier-coller dans la réponse
 - Mais apprentissage quasi-nul
- Objectif: MSDN, et uniquement MSDN

Mais alors, comment faire ?

- Examiner la liste de propositions de l'éditeur de Visual Studio
 - En particulier quand on tape un point après le nom d'un objet (cf. prochain slide)
- Accéder à la doc détaillée sur le site MSDN en tapant le nom de la méthode, puis « F1 » une fois la souris dessus
- Ne pas forcément s'inquiéter de comprendre *toutes* les explications, mais prendre le temps de s'assurer qu'on comprend l'essentiel
- Voir en particulier
 - Liste des méthodes et attributs
 - Exemples
- Ne pas hésiter à aller visiter les pages décrivant les concepts liés

```
static void Main(string[] args)
{
    string Texte;
```

Texte.

- Aggregate<>
- All<>
- Any<>
- AsEnumerable<>
- AsParallel
- AsParallel<>
- AsQueryable
- AsQueryable<>
- Average<>

(extension) TResult IEnumerable<TSource>.Aggregate<TSource,TAccumulate,TResult>(TAccumulate seed, Func<TAccumulate,TSource,TAccumulate> func, Func<TAccumulate,TResult> resultSelector)
Applies an accumulator function over a sequence. The specified seed value is used as the initial accumulator value, and the specified function is used to select the result value.

Exceptions:
System.ArgumentNullException

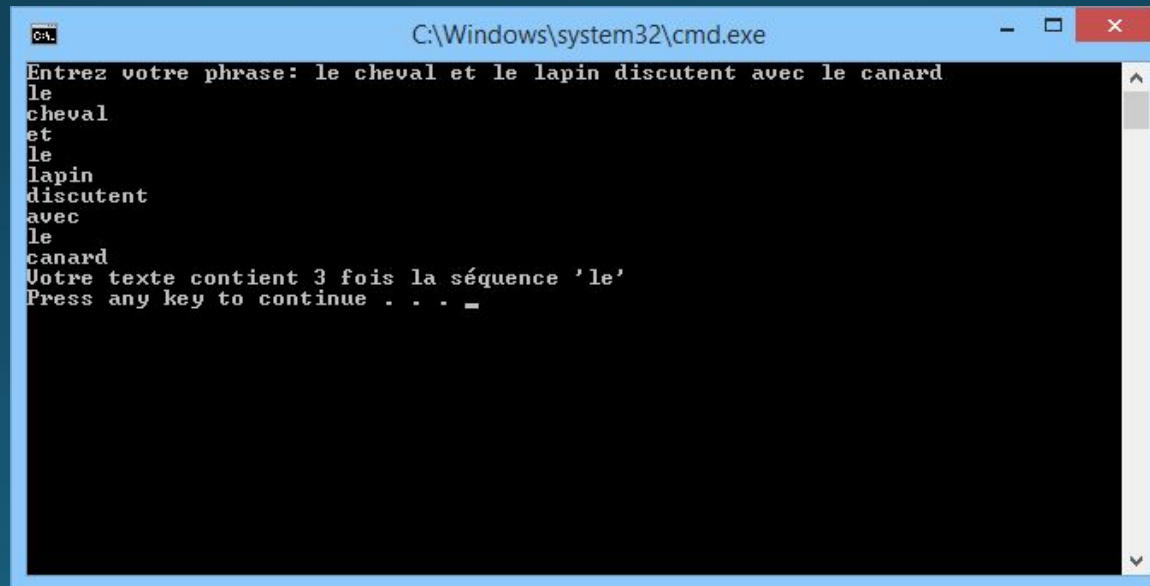
```
DateTi
DateTi
ime(1973,12,15); // Saisir clavier
Console.WriteLine("Aujourd'hui : " + Aujourd'hui.ToShortDateString());
```

Exercice : la classe String

- Ecrire un programme qui lit un texte au clavier et l'affiche en majuscules
- Ecrire un programme qui lit un texte au clavier et élimine tous les espaces tapés au début et à la fin du texte

Exercice : la classe String

- Ecrire un programme permettant de lire une phrase et de
 - La découper en mots, les mots étant séparés par des espaces dans la phrase initiale
 - Afficher combien de fois un groupe de lettres donné apparaît dans la phrase
 - Exemple



```
C:\Windows\system32\cmd.exe
Entrez votre phrase: le cheval et le lapin discutent avec le canard
le
cheval
et
le
lapin
discutent
avec
le
canard
Votre texte contient 3 fois la séquence 'le'
Press any key to continue . . . _
```

Exercice: la classe StringBuilder

- Ecrire un programme permettant de traduire un texte en « leet », c'est-à-dire en remplaçant
 - Les 'E' par des '3'
 - Les 'L' par des '7'
 - Les 'N' par '\N'
 - Etc.



The screenshot shows a web application for converting text to Leet Speak. It features three columns of input fields for mapping letters to their Leet equivalents. The first column contains mappings for A through M, the second for N through Z, and the third for special characters and numbers. Each mapping consists of a letter followed by an equals sign and a text input field. To the right of these columns are two buttons: 'Save' and 'Load'. At the bottom, there is a URL link to a sandbox website.

A=	4	N=	\N	Ä=	A
B=	13	O=	0	Ö=	O
C=	(	P=	>	Ü=	U
D=	[]	Q=	0,	1=	1
E=	3	R=	2	2=	2
F=	=	S=	5	3=	3
G=	6	T=	7	4=	4
H=	H	U=	[L]	5=	5
I=		V=	V	6=	6
J=	.]	W=	\W/	7=	7
K=	<	X=	{X}	8=	8
L=	1	Y=	Y	9=	9
M=	Y	Z=	2	0=	0

<http://sandbox.yoyogames.com/games/73538-leet-speak-converter>

Exercice: la classe DateTime

- Ecrire un programme qui
 - Affiche la date et l'heure
 - Demande à l'utilisateur sa date de naissance
 - Affiche quel jour de la semaine il est né
 - Affiche son âge exact (jours, heures, minutes, secondes)
 - En tenant compte des années bissextiles
 - Enfin, affiche le temps qu'il a fallu au programme pour calculer toutes ces informations
- Quelques mots-clefs
 - Classe DateTime, classe TimeSpan, Today, Now

Exercice: la classe Math

- Ecrire un programme qui calcule le sinus et le cosinus d'un nombre

Bonus:

- Il paraît que, pour tout x
$$\sin^2(x) + \cos^2(x) = 1$$
- Vérifiez-le sur une centaine d'exemples
- Que constate-t-on ? Pourquoi ?

Espaces de noms (namespaces)

Conflits de noms

- Un programme peut contenir un très grand nombre de classes
 - Certaines importées depuis d'autres programmes
- Que se passe-t-il si plusieurs portions de code ont choisi les mêmes noms pour des classes ?
 - P.ex. 2 classes, totalement différentes, appelées toutes deux « Saisie »
- Va provoquer un conflit à la compilation
 - Pénible à gérer
 - N'encourage pas la réutilisation de code
- Les **espaces de noms** (**namespaces**) permettent d'éviter ce problème

Espaces de noms

- Permettent d'isoler des portions de code
 - Deux namespaces différents ne se « voient » pas entre eux
 - Peuvent utiliser les mêmes noms
- Permettent également d'organiser son code
 - Définir des hiérarchies, avec des namespaces et des sous-namespaces
 - Similaire à l'idée des répertoires (directories) dans un système de fichiers
 - Mais il s'agit bien de 2 choses différentes !
 - Pas besoin que les fichiers du code soient structurés en répertoires pour organiser des namespaces

Exemple

(un peu tiré par les cheveux)

```
namespace Namespace1 {  
    class Machin {  
        public int truc;  
    }  
}
```

Au sein de Namespace2, Namespace1 est ignoré

```
namespace Namespace2 {  
    class Machin {  
        public string bidule;  
    }  
    class Program {  
        static void Main(string[] args) {  
            Machin m = new Machin();  
            m.bidule = "Hello";  
        }  
    }  
}
```

On peut donc y définir une classe Machin sans s'inquiéter qu'une autre porte le même nom ailleurs

m est bien une instance de « notre » classe Machin et non de l'autre

Accès à un autre namespace

- Il est parfois nécessaire d'accéder à un élément d'un autre namespace
- On le fait en utilisant le nom complet (*fully qualified name*) de l'élément
 - C'est-à-dire en précisant le(s) namespace(s), puis le nom de l'élément, séparés par des points
 - Exemples : Namespace1.Machin

Exemple

```
namespace Namespace1 {  
    class Machin {  
        public int truc;  
    }  
}  
  
namespace Namespace2 {  
    class Machin {  
        public string bidule;  
    }  
    class Program {  
        static void Main(string[] args) {  
            Machin m = new Machin();  
            m.bidule = "Hello";  
            Namespace1.Machin m2 = new Namespace1.Machin();  
            m2.truc = 7;  
        }  
    }  
}
```

Pour m2, on parle bien d'une instance de la classe Machin de Namespace1

Attention à bien le préciser à chaque accès : ici, on veut évidemment le constructeur de la classe définie dans Namespace1

Hiérarchie de namespaces

- Les namespaces peuvent être organisés en hiérarchie
 - = « un namespace dans un namespace »
- 2 manières de le faire
 - Par insertion directe

```
namespace Namespace3 {  
    namespace Namespace31 {  
    }  
}
```

- En utilisant plusieurs noms séparés par des points

```
namespace Namespace3.Namespace31 {  
}
```

- Les 2 méthodes sont équivalentes

Directive using

- Si l'on doit beaucoup utiliser le même namespace, les noms complets deviennent fastidieux
- La directive using permet de simplifier les choses
 - Exemple

```
using System.Text;  
using System.Threading.Tasks;
```

- Permet d'utiliser tous les éléments des namespaces System.Text et System.Threading.Tasks sans devoir préciser leurs noms complets
- VS ajoute automatiquement plusieurs de ces directives en début de code

Remarque

- Namespaces et fichiers sont 2 concepts différents
 - Le code écrit dans plusieurs fichiers différents peut faire partie du même namespace
 - C'est d'ailleurs le cas par défaut, quand VS définit une nouvelle classe dans un fichier séparé

En pratique

- Les classes prédéfinies de C# sont largement organisées en hiérarchies de namespaces
- Nous rencontrerons donc souvent des références à des noms complets, etc.

Gestion des erreurs

▪
▪

try et catch

Motivation

- Que se passera-t-il si on exécute le code suivant ?

```
static void Main(string[] args) {  
    int i = 5, j = 0;  
    int res = i / j;  
    Console.WriteLine("Résultat: " + res);  
}
```

- Le programme va s'arrêter brutalement sur un message d'erreur
- Cela peut aussi se produire dans des cas que l'on ne peut pas contrôler (valeurs tapées par l'utilisateur...)
- Un programme de qualité professionnelle ne doit pas « planter » de cette façon

try/catch

- Permet de prendre la main quand de telles erreurs se produisent, afin de
 - Traiter l'erreur
 - Revenir à une situation stable
 - Sans arrêter le programme
- `/!\` A réserver aux situations exceptionnelles
 - Le but n'est pas de faire n'importe quoi et compter sur les try/catch pour rattraper les choses

Principe de base

- Le bloc d'instructions susceptible de poser problème est placé dans un bloc try
- Ce bloc est suivi d'un bloc catch
- Dès qu'une erreur se produit
 - Dans l'une des instructions du bloc try
 - Ou dans l'une des méthodes appelées depuis ce bloc
- le programme passe immédiatement au bloc catch, en lui passant un objet de classe Exception

Exemple

```
try
{
    int i = 5, j = 0;
    int res = i / j;
    Console.WriteLine("Résultat: " + res);
}
catch (Exception e)
{
    Console.WriteLine("Houston, We've Had a Problem");
}
Console.WriteLine("Et on continue ici");
```

Try/catch

- Dès qu'une erreur survient dans le bloc try
 - L'exécution de ce bloc est interrompue
 - Un objet de classe Exception (ou une classe dérivée) est créé
 - Le contrôle passe au bloc catch, qui reçoit l'exception
- Si aucune erreur ne se produit
 - Le bloc try est exécuté normalement
 - Le bloc catch n'est pas exécuté
- Dans tous les cas
 - L'exécution se poursuit ensuite en passant à ce qui suit les 2 blocs

Exercice

Ecrivez un programme qui

- Demande à l'utilisateur d'entrer un mot
- Affiche la 10^{ème} lettre de ce mot
- Génère et traite une exception si le mot entré est trop court

Modifiez votre programme pour que

- La partie du code qui provoque l'erreur se trouve dans une méthode appelée depuis le programme principal
- Mais les try/catch soient toujours dans le programme principal
- Le try/catch fonctionne-t-il encore ?

Exemple 2: appel de méthode

```
try
{
    int i = 5, j = 0;
    int res = g(i,j);
    Console.WriteLine("Résultat: " + res);
}
catch (Exception e)
{
    Console.WriteLine("Houston, We've Had a Problem");
}
```

L'erreur sera aussi interceptée si elle survient dans l'exécution de g()

Exception depuis une sous-routine

- Dès que l'erreur survient,
(ici, dans la méthode `g()`)
- le système génère une exception et cherche un bloc pouvant la traiter
- Si aucun bloc de ce type n'est présent, le système « remonte » l'exception vers la méthode appelante
- ... et ainsi de suite
- En dernier recours, le système prend les choses en main et met fin au programme

La classe exception

- Permet de créer des objets « exception », fournissant des informations sur la nature de l'erreur
 - Définit divers attributs, parmi lesquels
 - Message : un message décrivant le problème
 - Source: le nom de l'application ou de l'objet ayant causé l'erreur
 - ...
 - Possède diverses sous-classes
 - ArithmeticException
 - IndexOutOfRangeException
 - ...
- Le type de l'erreur fournit donc lui aussi de l'information sur l'erreur

Le type de l'erreur fournit de l'information

```
static void TryCatch5()
{
    int i = 7, j = 0; // Tester avec d'autres valeurs de i,j
    int[] array = new int[3];
    try {
        array[i] = 42 / j;
    }
    catch (Exception e) {
        if (e is ArithmeticException)
            Console.WriteLine("Erreur sur opération arithmétique");
        else if (e is IndexOutOfRangeException)
            Console.WriteLine("Accès en dehors des bornes");
        else Console.WriteLine("Problème indéterminé");
    }
}
```

Clauses catch multiples

- Plusieurs clauses catch peuvent suivre un try
 - Avec différents types d'erreur en argument

```
catch (ArithmeticException e) { [...] }  
catch (IndexOutOfRangeException e) { [...] }  
catch (Exception e) { [...] }
```

- Le contrôle est passé au premier bloc dont le type convient
- Tous les autres (avant et après) sont ignorés

Clauses catch multiples: exemple

```
static void TryCatch4()
{
    int i = 7, j = 5; // Tester avec d'autres valeurs de i,j
    int[] array = new int[3];
    try {
        array[i] = 42 / j;
    }
    catch (ArithmeticException e) {
        Console.WriteLine("Erreur sur opération arithmétique");
    }
    catch (IndexOutOfRangeException e) {
        Console.WriteLine("Accès en dehors des bornes");
    }
    catch (Exception e) {
        Console.WriteLine("Problème indéterminé");
    }
}
```

Clauses catch multiples

- Plusieurs clauses catch peuvent suivre un try
 - Avec différents types d'erreur en argument

```
catch (ArithmeticException e) { [...] }  
catch (IndexOutOfRangeException e) { [...] }  
catch (Exception e) { [...] }
```

- Le contrôle est passé au premier bloc dont le type convient
- Tous les autres (avant et après) sont ignorés
- **/!\ Attention à l'ordre !**
 - Que se passerait-il si la dernière ligne ci-dessus était placée en premier ?

Finally

- Un bloc finally peut aussi être spécifié
 - Sera exécuté de toutes façons
 - Qu'il y ait eu erreur (exécuté après le bloc catch)
 - Ou non (exécuté après le bloc try)
 - Permet de libérer des ressources, ...

Génération d'exceptions

- Il est également possible de définir ses propres exceptions
- Pour cela
 - Définir une nouvelle sous-classe héritée de la classe Exception
 - Il n'est pas forcément nécessaire d'y ajouter de nouvelles méthodes ou attributs
 - Générer une exception avec l'instruction throw

Génération d'exceptions

```
class MaPropreException : Exception {  
    public MaPropreException(string msg) : base(msg) {}  
}  
static void TryCatch6() {  
    try {  
        Console.WriteLine("Entrez votre âge");  
        int age = Int32.Parse(Console.ReadLine());  
        if (age < 18) throw new MaPropreException("Ce programme est  
interdit aux moins de 18 ans");  
    }  
    catch (MaPropreException e) {  
        Console.WriteLine(e.Message);  
    }  
}
```

Limites de try/catch

```
try {  
    int i = 5;  
    for (int j = 1; j < 10; j++) {  
        int res = i / (j - 6);  
        Console.WriteLine("Résultat: " + res);  
    }  
}  
catch (Exception e) {  
    Console.WriteLine(e.Message);  
}
```

Exercice

- Le programme ci-dessus va générer une erreur en $j == 6$
 - Que se passera-t-il alors ?
 - Et après ?

Limites de try/catch

- try/catch permet d'intercepter l'exception et de rendre le contrôle au programmeur
- Mais fournit relativement peu d'outils pour
 - Identifier l'endroit exact où l'erreur s'est produite
 - Reprendre le cours normal de l'exécution après traitement

□ Cette tâche incombe au programmeur !

- Dans le cas présent, il devrait par exemple
 - Déterminer à quelle étape le programme était arrivé
 - Reprendre l'exécution après cette étape
- (ou reprendre à zéro, ou demander de nouvelles données, ou libérer les ressources et abandonner ce traitement, ou ...)

try/catch : bonnes pratiques

- Attention à garder le code lisible
 - Utilisation limitée de try/catch
 - Sur des blocs d'instructions réduits
 - Traitement aussi simple que possible
- Souvent beaucoup plus simple d'anticiper et d'empêcher l'erreur que de récupérer a posteriori
 - Tester si les valeurs entrées sont conformes
 - Lire les valeurs de statut renvoyées par les méthodes
(p.ex. TryParse() renvoie un booléen indiquant si la conversion a réussi)
- ...

Exercice

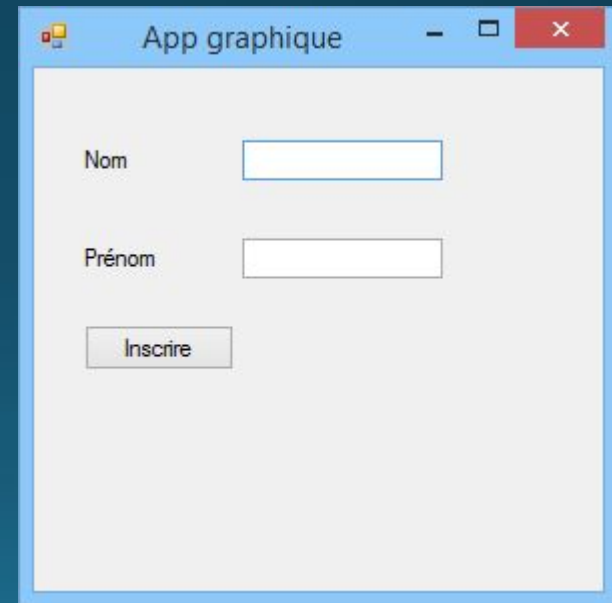
Ecrivez un programme qui

- Demande 5 nombres à l'utilisateur, et remplit un tableau T avec ces nombres
- Demande ensuite deux valeurs i et j et affiche la valeur de $T[i] / T[j]$
- Réagit à tous les problèmes pouvant survenir, en affichant des messages *différents* selon le type de problème
- Supposons qu'il soit également interdit d'accéder au premier nombre du tableau : définissez votre propre exception pour traiter ce cas

Développement d'applications graphiques

Fenêtres

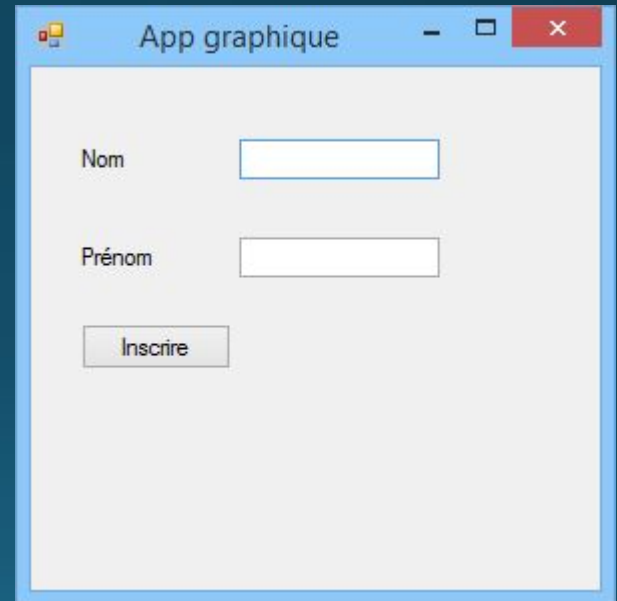
- Un composant fondamental d'une application graphique:
Les **fenêtres** (**windows**), ou **formulaire** (**forms**)
- Du point de vue de l'utilisateur
 - Un programme se résume souvent à une ou plusieurs fenêtres avec lesquelles il peut interagir



Objets fenêtres et composants

Du point de vue du programmeur

- Chaque élément est un **objet** d'une classe graphique
 - Button, Label, ListBox ...
- Chacune de ces classes définit
 - Des propriétés : Font, Size ...
 - Des méthodes : Show(), Close() ...
 - Des événements auxquels réagir : Click, KeyPress ...



Programmation en mode console vs. programmation événementielle

En mode « console », le programmeur est aux commandes des interactions avec l'utilisateur

- Choisit
 - Quels inputs l'utilisateur doit fournir, et quand
 - Quand afficher les résultats

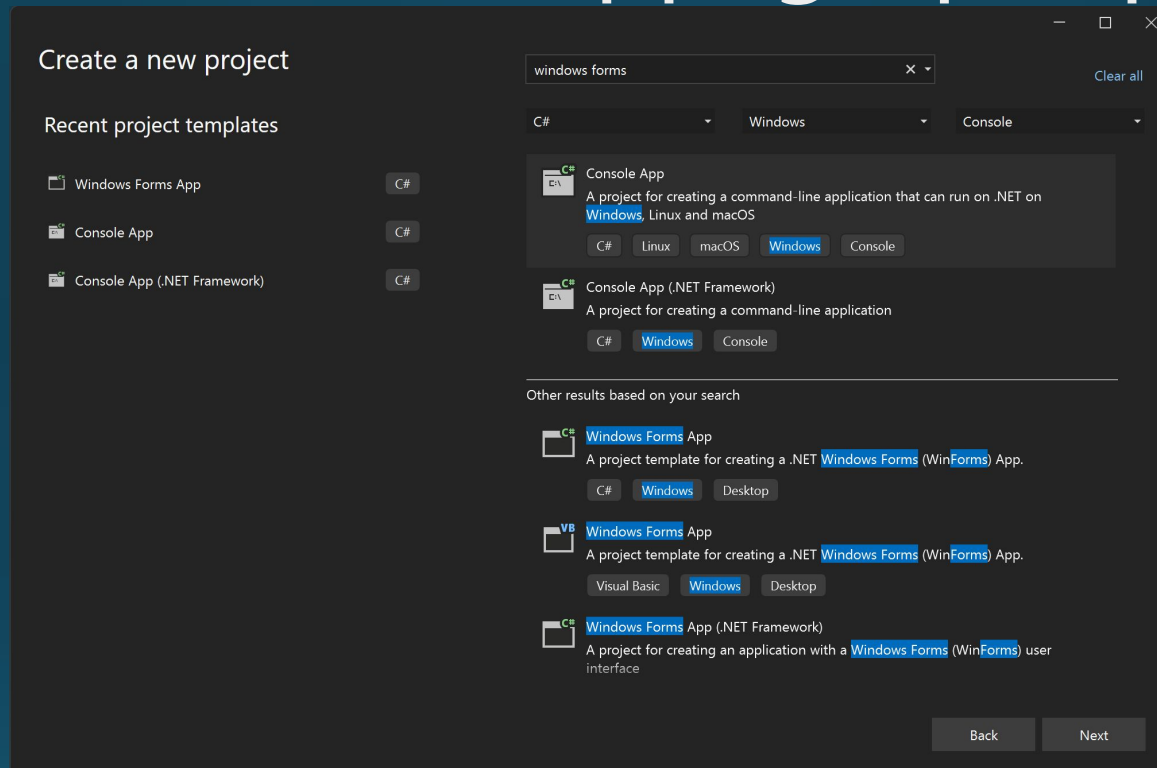
En mode « fenêtre », l'utilisateur est aux commandes

- Décide
 - De remplir tel ou tel champ
 - De cliquer sur un bouton
 - ...
- Le programme doit être prêt à réagir à ces événements

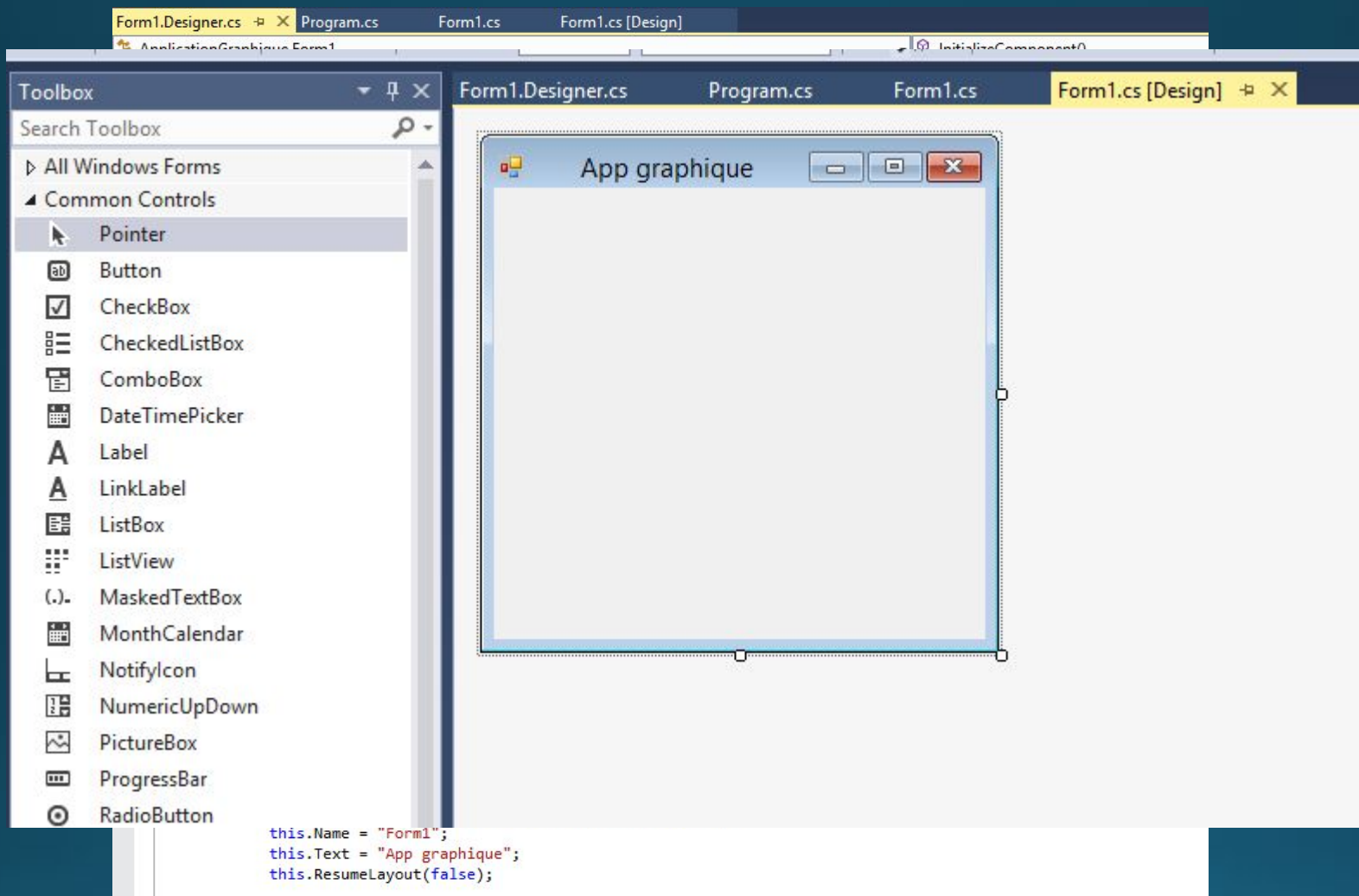
Programmation événementielle

- Le programme est « en attente » des actions de l'utilisateur
 - Définit des méthodes à appeler en réponse à ces actions (événements)
- Peut sembler plus simple...
 - « Y a qu'à prévoir une méthode par bouton »
- ... mais ce n'est qu'une impression !
 - Que faire si l'utilisateur n'a pas fourni toutes les données ?
 - Que faire s'il modifie une donnée déjà introduite ?
 - Quand/comment recalculer ?
 - ...
- Demande beaucoup plus de **flexibilité** et d'**anticipation**

Création d'une app. graphique



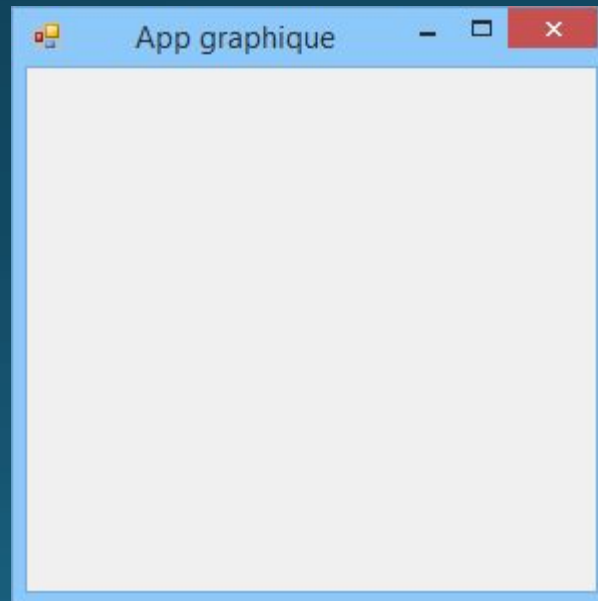
- Sélectionner/chercher "Windows Forms App"
- Remarques:
 - Windows Form vs. WPF: cf. <https://www.educba.com/winforms-vs-wpf/>



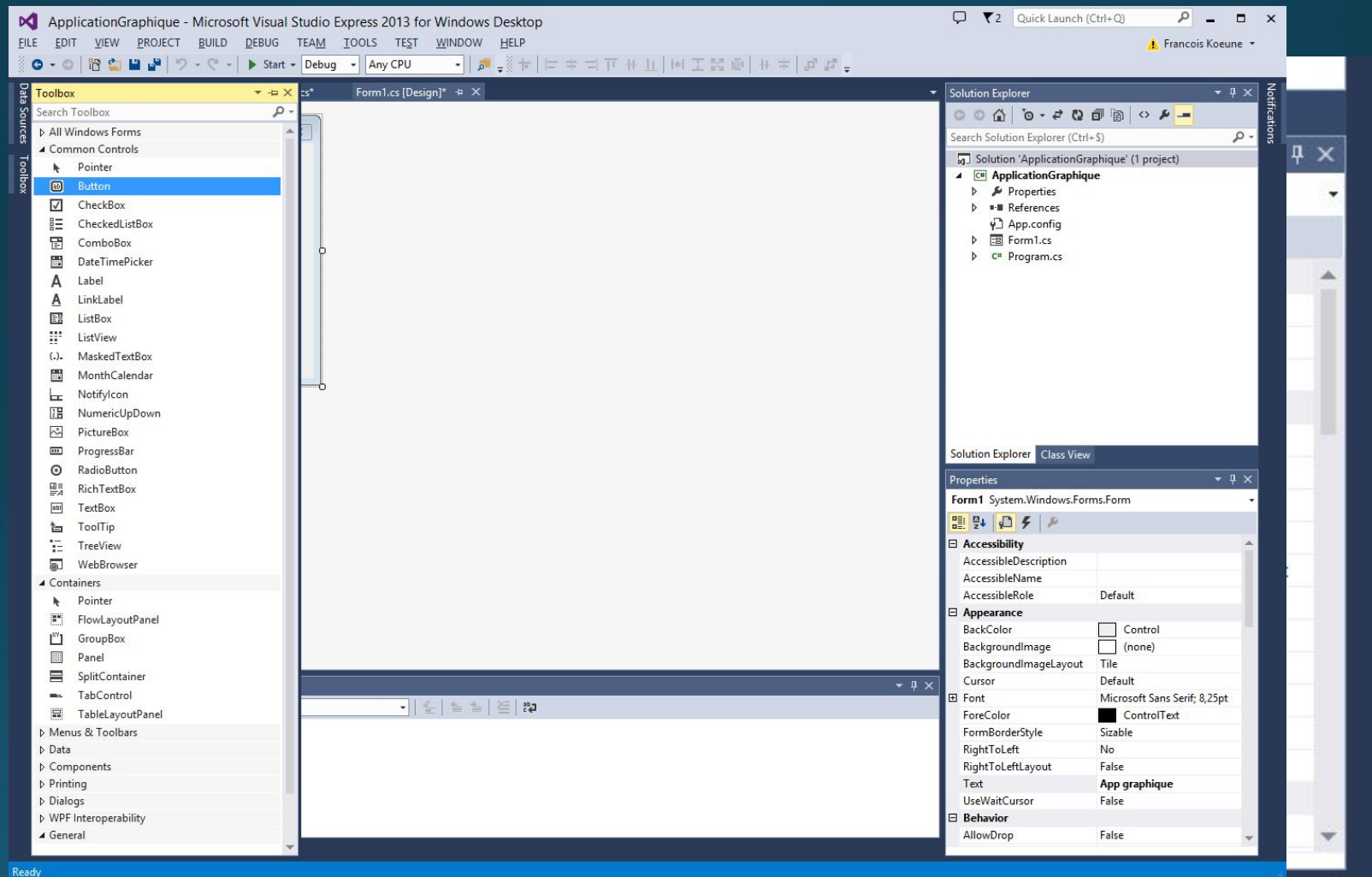
- Le programmeur peut se concentrer sur le code spécifique à son application

Création d'une app. graphique

- L'application « vide » ainsi créée peut déjà être exécutée
- On peut déplacer, minimiser, fermer la fenêtre...



Edition des éléments graphiques

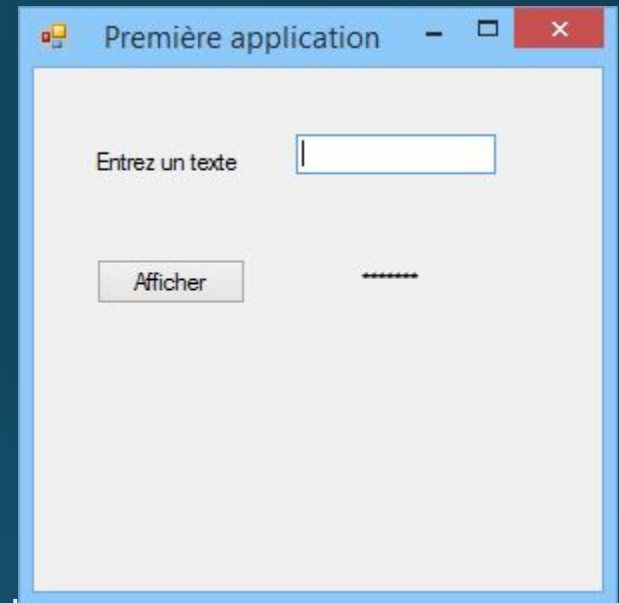


Quelques composants utiles...

- Label : permet d'afficher un texte informatif
- TextBox : zone de saisie d'input utilisateur
- Button : déclenchement d'action par l'utilisateur

Exercice

- Créez un formulaire contenant
 - Deux labels
 - Une zone de saisie
 - Un bouton
- Modifiez leurs paramètres pour que
 - Le titre de la fenêtre principale soit « Première application »
 - Le nom (**pas le texte affiché**) de la zone d'éditation soit ZoneSaisie
 - Le texte du 1^{er} label soit « entrez un texte »
 - Le texte du 2^{ème} label soit « **** »
 - Le nom du 2^{ème} label soit « Résultat »
 - Le nom du bouton soit BoutonAfficher
 - Le texte du bouton soit « Afficher »



Pourquoi renommer les composants ?

- Visual Studio attribue des noms par défaut aux composants: `textBox1`, `textBox2`, `button1` ...
- On se « perd » facilement dès que le nombre de contrôles augmente
 - Quel est le rôle de `button19` ?
- Comme pour toutes les variables, il est beaucoup plus ergonomique de donner des noms explicites
 - `BoutonConfirmer`, `TextBoxPrénom`, ...
- On recommande parfois que le nom indique aussi de quel type de contrôle il s'agit
 - `BoutonConfirmer`, ou `b_Confirmer`

Mode édition et code

- A mesure que des éléments sont ajoutés, Visual Studio génère automatiquement le code correspondant dans `Form1.Designer.cs`
- Bien entendu,
 - Il est également possible de construire des formulaires directement via du code
 - Ceci permet des formulaires dynamiques, dont le contenu dépend de ce qui se passe durant l'exécution
- `/!\` Certaines parties du code généré automatiquement seront écrasées, même si on les édite « à la main »

Mode édition et code

- En examinant ce code, on voit qu'on est en train de définir
 - Une classe Form1
 - Qui a pour attributs les différents éléments graphiques ajoutés

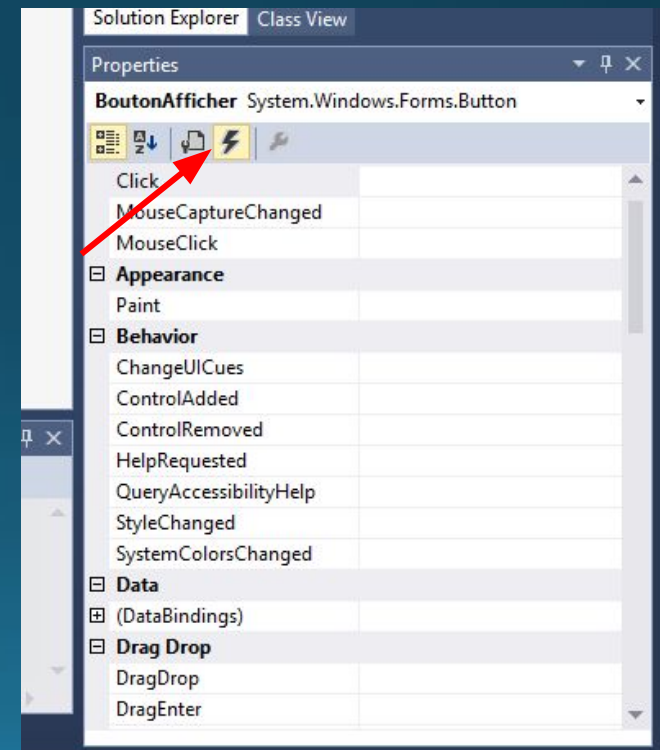
□ C'est exactement le même principe que quand nous définissons

- Une classe Personne
- Possédant un attribut CompteBancaire

```
partial class Form1    {  
    /// <summary>  
    /// Required designer variable.  
    [...]  
    private System.Windows.Forms.Label label1;  
    private System.Windows.Forms.TextBox ZoneSaisie;  
    private System.Windows.Forms.Button BoutonAfficher;  
    private System.Windows.Forms.Label Résultat;
```

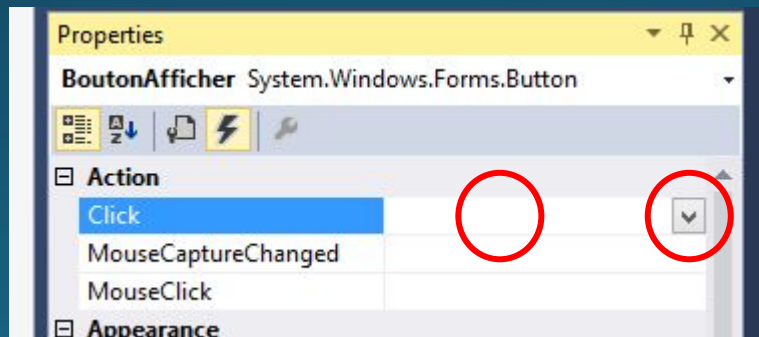
Réagir à un événement

- Nous allons maintenant compléter notre programme pour qu'il fasse quelque chose quand on clique sur le bouton
- Pour cela,
 - Sélectionner le bouton
 - Aller dans l'onglet événements de ses propriétés
 - On y voit les principaux événements qui peuvent affecter ce contrôle
 - Celui qui nous intéresse est « Click »
 - Événement déclenché à chaque fois que l'utilisateur clique sur le bouton



Réagir à un événement

- En sélectionnant l'événement « Click », on peut définir la méthode à appeler quand l'événement se produit
- On peut
 - Utiliser le menu déroulant (pour le moment vide) pour choisir une méthode existante
 - Double-cliquer pour que VS crée automatiquement une nouvelle méthode
- VS nous envoie dans la fenêtre d'édition du code



Réagir à un événement : le code

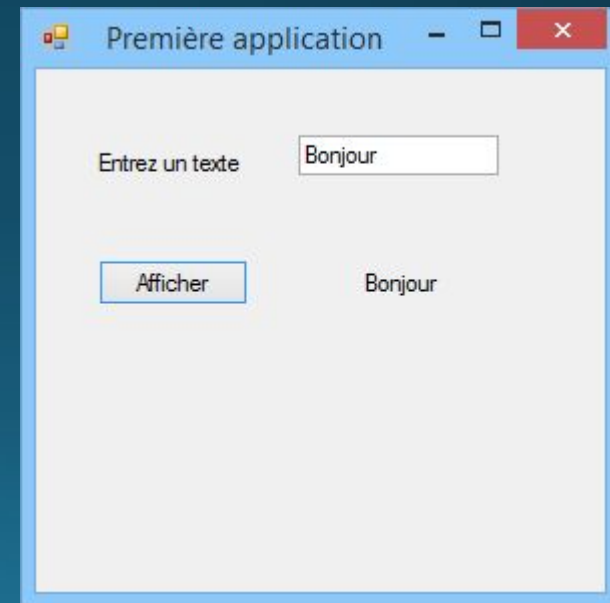
```
public partial class Form1 : Form {  
    public Form1() {  
        InitializeComponent();  
    }  
    private void BoutonAfficher_Click(object sender, EventArgs e) {  
    }  
}
```

- Notons que

- Le nom donné suit le schéma NomComposant_NomÉvénement
- La méthode BoutonAfficher_Click() est une méthode membre de la classe Form1 de notre formulaire
- Elle reçoit 2 paramètres
 - sender : quel objet a provoqué l'événement (ici, BoutonAfficher)
 - e : contient des informations sur l'événement
 - Dans ce cas-ci, e ne contient rien
 - Pour un déplacement souris, contiendrait la position de la souris ...

Réagir à un événement : le code

- Programmons la méthode BoutonAfficher_Click() pour qu'elle affiche dans le 2^{ème} label le texte que l'utilisateur aura entré dans la zone de saisie
- En fait très simple à faire:
 1. Récupérer le texte de ZoneSaisie
 - ZoneSaisie est un objet de classe TextBox
 - Cette classe définit une propriété Text
 2. Placer ce texte dans la zone texte de Résultat
 - Résultat est un objet de classe Label
 - Cette classe définit elle aussi une propriété Text



Réagir à un événement : le code

```
private void BoutonAfficher_Click(object sender, EventArgs e)
{
    string s = ZoneSaisie.Text;
    Résultat.Text = s;
}
```

- On peut bien évidemment aussi le faire en une ligne, sans passer par la variable s

□ De manière générale, écrire du code dans ces méthodes n'est pas différent de l'écriture de code « classique »

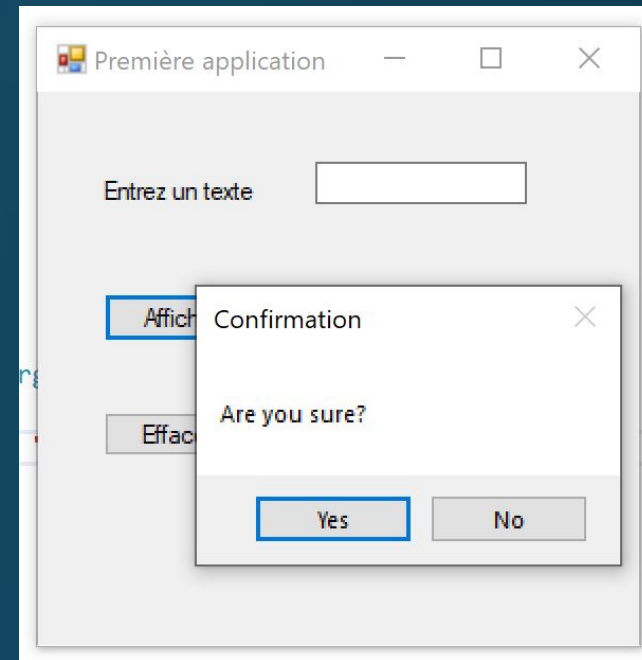
- Boucles, if, variables...
- On peut aussi ajouter ses propres attributs à la classe

Exercice

- Complétez l'application en
 - Modifiant BoutonAfficher_Click() pour qu'elle
 - Affiche le texte en majuscules si l'utilisateur a tapé un texte
 - Affiche « Saisie vide » si l'utilisateur n'a rien tapé
 - Ajoutant un bouton « BoutonEffacer » qui efface le contenu de la zone de saisie (mais pas celui du label résultat)

La classe MessageBox

- Une classe très utile pour afficher des messages à l'utilisateur et collecter un feedback
- Sa principale méthode : **show**
 - Méthode statique
 - Surchargée de nombreuses fois (= « existe en beaucoup de versions »)
 - Exemples
 - `public static DialogResult Show(string text);`
 - `public static DialogResult Show(string text, string caption, MessageBoxButtons buttons, MessageBoxIcon icon);`
 - Renvoie le feedback utilisateur sous la forme d'un objet `DialogResult`



La classe MessageBox

- `public static DialogResult Show(string text);`
 - Affiche text dans une boîte de dialogue minimaliste
- `public static DialogResult Show(string text, string caption, MessageBoxButtons buttons, MessageBoxIcon icon);`
 - Version plus complète, avec
 - `string text` : le texte à afficher
 - `string caption` : le titre de la fenêtre
 - `MessageBoxButtons buttons` : les boutons « feedback » : OK, OKCancel, YesNo...
 - `MessageBoxIcon icon` : l'icône à afficher : Error, Question, Warning...
- Et 19 autres variantes !

Remarque

- En C#, il arrive régulièrement qu'on doive manipuler des constantes pré-définies
 - Des couleurs : Red, Blue, Green...
 - Des boutons standards : YesNo, OKCancel...
 - Des icônes fréquentes : Question, Warning, Error, ...
 - ...
- Pour faciliter le traitement, C# a défini des classes regroupant ces valeurs
 - (En fait, pas exactement des classes, mais peu importe)
 - Par exemple
 - Pour accéder à la couleur rouge : `Color.Red`
 - Pour obtenir une icône « warning » : `MessageBoxIcon.Warning`
 - Pour comparer et tester si la valeur de retour est « Yes » : `DialogResult.Yes`

MessageBox : exemple

```
DialogResult Reponse;  
Reponse = MessageBox.Show("Continuer le traitement?", "Confirmer",  
    MessageBoxButtons.OKCancel, MessageBoxIcon.Warning);  
if (Reponse == DialogResult.OK)                // Continuer le traitement  
    [...]
```

Exercice

- Ajoutez un bouton « b_MessageBox » à votre application
- Lorsque l'utilisateur clique dessus, le programme teste si quelque chose a été entré dans ZoneSaisie
- Si ce n'est pas le cas
 - Il affiche une MessageBox « Etes-vous sûr de ne rien vouloir saisir? » avec des boutons Oui/Non
 - Si l'utilisateur choisit « Non », il affiche une nouvelle MessageBox « Il me semblait bien »
 - S'il choisit « Oui », il affiche une MessageBox « OK: c'est vous qui voyez »

Enabled et Select

- Communes à un grand nombre de contrôles (Button, TextBox, ComboBox...)
 - **Enabled** permet d'activer/désactiver un contrôle (ou de tester son état)

```
button_Ajouter.Enabled = false; // Ce bouton devient inutilisable
```

- **Select()** permet de sélectionner le contrôle visé

```
textBox_Saisie.Select(); // Le focus passe sur ce contrôle
```

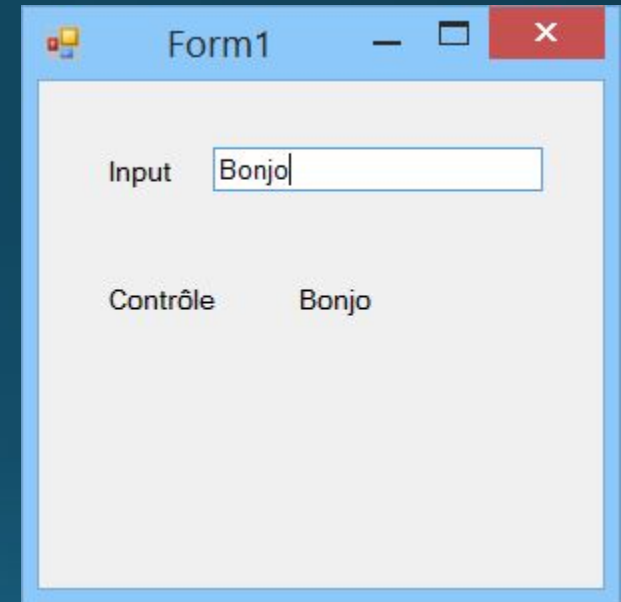

Quelques contrôles courants

Label et TextBox

- Les 2 composants d'affichage et affichage/saisie
- Propriété principale : Text (en lecture/écriture)
- De nombreuses autres propriétés existent
 - Police de caractères, alignement texte, ...
- Peut également réagir à divers événements
 - Click : quand l'utilisateur clique sur le composant
 - ...
 - Un événement de TextBox souvent utile : **TextChanged**
 - Se produit à chaque fois que le champ de saisie évolue

Exercice

- Ecrivez une application comportant
 - Une zone de saisie
 - Un label « Contrôle »
- Telle que le label affiche le contenu de la zone de saisie au fur et à mesure de la frappe
- Bonus : faites en sorte que la couleur du label de contrôle bascule entre le rouge et le noir lorsqu'on clique dessus



TextBox multilignes

- Une TextBox peut afficher plusieurs lignes de texte
- Quelques propriétés liées au multiligne
 - Multiline : multiligne ou non
 - ScrollBars : barres de défilement
 - AcceptReturn : si True, la touche Entrée fait passer à la ligne
 - AcceptTab : si True, la touche Tab insère une tabulation

Exercice

- Ajoutez à l'application précédente
 - Un bouton Ajouter
 - Une TextBox multilignes
- Telles que
 - Quand on clique sur le bouton, le texte de la première TextBox soit ajouté à la seconde, sur une nouvelle ligne
- Remarque
 - Pour insérer un retour à la ligne
 - Caractère `\n`
 - Ou, mieux, `System.Environment.NewLine`

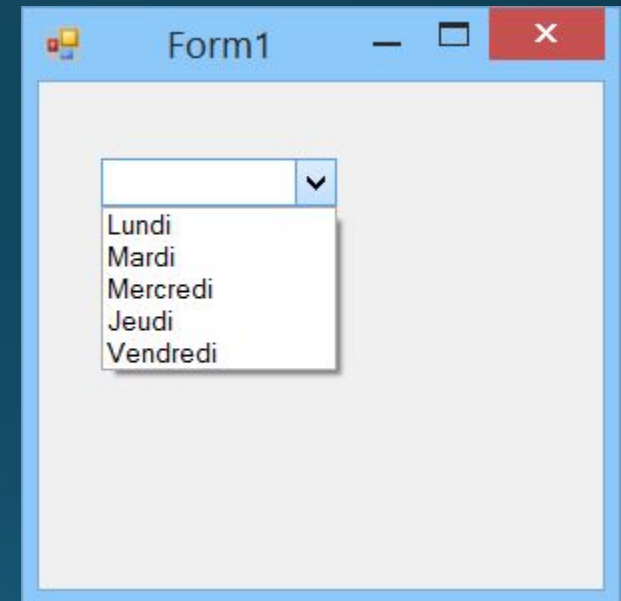
The screenshot shows a Windows application window titled "Form1". Inside the window, there is a label "Input" followed by a single-line text box containing the text "Dag". To the right of this text box is a button labeled "Ajouter". Below these elements, there is a label "Contrôle" followed by a multi-line text box. This multi-line text box contains three lines of text: "Hello", "Bonjour", and "Ciao". The multi-line text box has a vertical scrollbar on its right side.

Label multilignes

- Un label n'a pas de propriété Multiline
(par défaut, il est multilignes)
- Pour afficher son contenu sur plusieurs lignes
 - Soit insérer des retours à la ligne dans le texte
 - Soit définir une largeur maximale via la propriété MaximumSize
 - Le texte sera automatiquement réorganisé

Listes déroulantes ComboBox

- Liste déroulante avec zone de saisie
 - Permet de choisir parmi plusieurs options, ou d'en introduire une nouvelle
- 3 sortes de ComboBox (propriété **DropDownStyle**)
 - Simple
 - Pas de liste déroulante, zone d'édition
 - DropDown
 - Liste déroulante, zone d'édition
 - DropDownList
 - Liste déroulante, pas de zone d'édition



ComboBox : quelques autres méthodes et propriétés

- **Items** : contient la liste des éléments
 - Peut être remplie
 - Statiquement, via l'interface Visual Studio
 - Dynamiquement, via du code
- Accès/remplissage ("comme toute liste")
 - **Items.Count** : nombre d'éléments
 - **Items.Add**(object o) : ajoute o en fin de liste
 - **Items.AddRange**(object[] o) : ajoute un tableau d'objets à la liste
 - **Items.Insert**(int i, object o) : ajoute o en position i
 - **Items.RemoveAt**(int i) : enlève l'élément en position i
 - **Items.IndexOf**(object o) : position de o dans la liste
 - **SelectedItem** : élément sélectionné
 - **Text** : texte de l'élément sélectionné
 - ...

Remarque

- Un élément d'une ComboBox est de type object
 - Pas nécessairement une simple chaîne de caractère
 - Peut être n'importe quel objet
 - Il est par exemple possible d'utiliser une ComboBox pour sélectionner directement un Héros, une Personne, une StationService, ... !
 - Le texte à afficher est alors régi par la méthode ToString()

ComboBox : événements

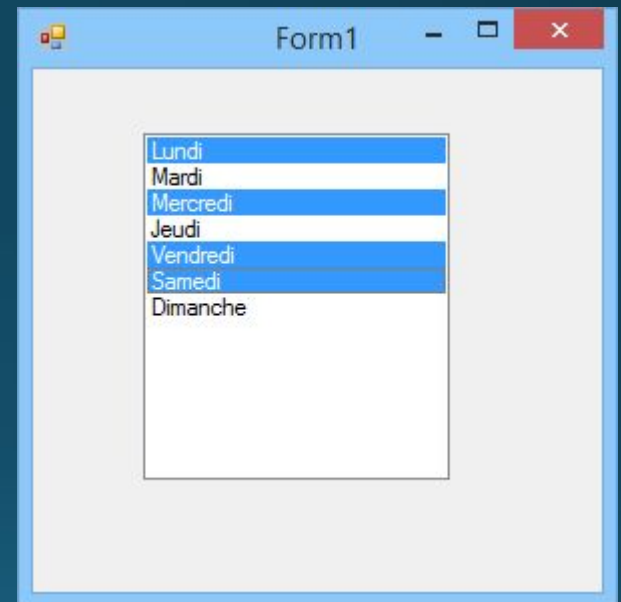
- Événement principal : **SelectedIndexChanged**
 - Se produit à chaque fois que l'utilisateur sélectionne un élément
 - Même si c'est le même qu'avant

Exercices

- Ecrire un programme qui invite un utilisateur à choisir le montant qu'il veut donner à une bonne œuvre : 1, 10, 100 ou 1000 euros
 - Pendant qu'il parcourt les choix, un label commente le montant : « OK, Cool, Waow, Génial »
 - Un bouton permet de confirmer le choix final. A la confirmation, un autre label affiche « Merci pour votre don de ... »
- Ecrire un programme offrant une barre de recherche « Google-like »
 - L'utilisateur peut choisir une ancienne valeur, ou en taper une librement
 - Lorsqu'il clique « Confirmer », la valeur tapée est ajoutée à la liste des choix

ListBox

- Similaire aux ComboBox, mais
 - Pas de zones d'édition
 - Possibilité de permettre des sélections multiples



ListBox: propriétés, méthodes et événements

- Plusieurs sont identiques à celles des ComboBox
 - Items.Count : nombre d'éléments
 - Items.Add(object o) : ajoute o en fin de liste
 - SelectedIndexChanged
 - ...
- Principale différence: sélection multiple
 - **SelectionMode** : type de sélection autorisée
 - One, MultiSimple (clic), MultiExtended (avec <shift> et <ctrl>)
 - **SelectedItems** : permet de récupérer les éléments sélectionnés
 - SelectedItems.Count : nombre d'éléments sélectionnés
 - SelectedItems[] : liste des éléments sélectionnés

Remarque (astuce) : les contrôles sont des objets comme les autres

- Il arrive que l'on doive répéter le même type de traitement sur plusieurs groupes de contrôles
 - (Cf. exemple prochain exercice)
- Un contrôle étant un objet, on peut
 - L'affecter à une variable
 - Le passer en paramètre
 - ...

□ Permet parfois d'éviter de copier-coller des blocs de code

```
private void Afficher(Personne P, Label l_Nom, Label l_Prénom)
{ [...] }

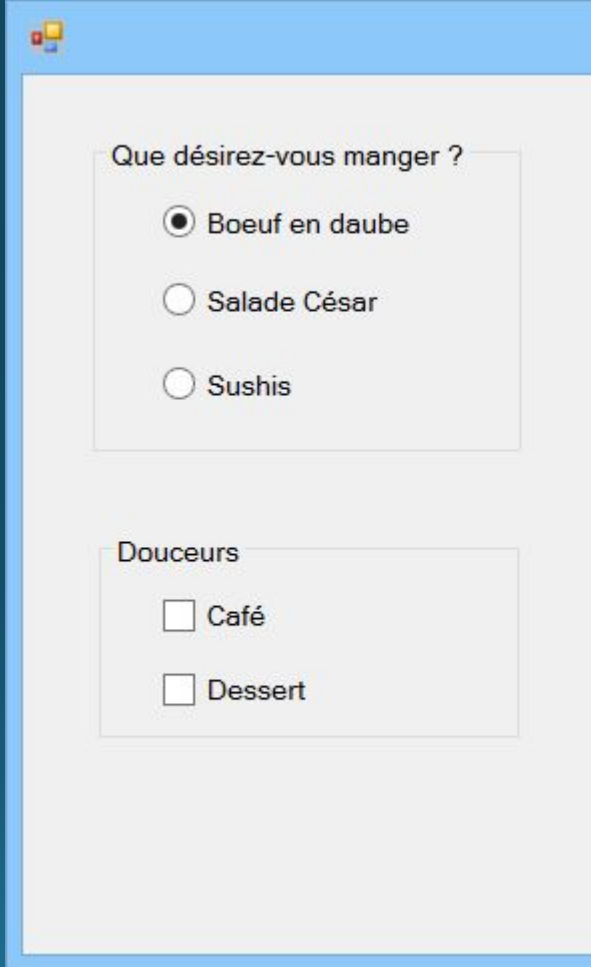
[...]
Afficher(p1, label_Nom1, label_Prénom1);
```

Exercice

- Ecrivez une application identique à l'application Exercice_ListBox.zip disponible sur le portail
- Remarques
 - Pour simplifier, commencez par la programmer en mode sélection simple, puis autorisez les sélections multiples
 - Les boutons « --> » et « <-- » ont des comportements très proches : pensez à définir une méthode générique prenant les contrôles en paramètres

GroupBox, CheckBox & RadioButton

- Utilisés pour la sélection d'options
 - **GroupBox** crée un groupe d'éléments
 - Remarque: GroupBox se trouve dans la série « Containers »
 - **CheckBox** permet de sélectionner une ou des options
 - **RadioButton** permet de choisir *une seule* option dans un groupe
- Exemple
 - Les RadioButtons permettent de prendre l'un des menus, au choix
 - Les CheckBoxes permettent de sélectionner café et/ou dessert



The screenshot shows a Windows application window with a light blue title bar. Inside the window, there is a form with two sections. The first section is titled "Que désirez-vous manger ?" and contains three radio buttons: "Boeuf en daube" (selected), "Salade César", and "Sushis". The second section is titled "Douceurs" and contains two checkboxes: "Café" and "Dessert".

GroupBox, CheckBox & RadioButton

- Éléments principaux
 - Événement **CheckChanged** : une case a été cochée/décochée
 - Propriété (booléenne) **Checked** : cette case est-elle cochée ?

```
private void checkbox_Café_CheckedChanged(object sender, EventArgs e) {  
    if (checkbox_Café.Checked) MessageBox.Show("Et un café");  
}
```

- Remarque
 - Une sélection de RadioButton provoque 2 événements **CheckChanged**
 - Au moment où le premier est désélectionné
 - Au moment où le deuxième est sélectionné

Exercice

- Implémentez cette interface

The screenshot shows a Windows application window titled "Form1". Inside the window, there are two main sections on the left and two text areas on the right.

Left Section 1: A group box titled "Que désirez-vous manger ?" contains three radio buttons: "Boeuf en daube", "Salade César", and "Sushis". The "Sushis" radio button is selected.

Left Section 2: A group box titled "Douceurs" contains two checkboxes: "Café" and "Dessert". The "Dessert" checkbox is checked.

Right Section 1: A text area labeled "Votre choix" displays "Sushis , dessert".

Right Section 2: A text area labeled "log" displays a list of selection events: "Boeuf en daube sélectionné", "Boeuf en daube désélectionné", "Salade César sélectionné", "Café sélectionné", "Dessert sélectionné", "Café désélectionné", "Salade César désélectionné", and "Sushis sélectionné".

- « Votre choix » affiche le choix courant
- « log » affiche tous les événements qui se sont produits
- Remarque
 - Définissez une méthode unique pour réagir aux événements des 3 RadioButtons et une méthode autre pour les 2 CheckBoxes (ou une méthode unique pour tous ces contrôles)

Vie et mort d'une application

Vie et mort d'une application

- 3 événements intéressants dans la « vie » d'un formulaire
 - **Load** : formulaire en cours de chargement
 - **FormClosing** : formulaire en cours de fermeture
(peut encore être annulé)
 - **FormClosed** : formulaire fermé
- Il est bien évidemment possible de programmer des méthodes réagissant à ces événements

Load, FormClosing, FormClosed : comment ça se passe ?

Exercice

- Ecrire une application qui affiche 3 MessageBox
 - Une « Load » au démarrage
 - Une « Closing » lorsque l'application reçoit l'ordre de se fermer
 - Une « Closed » à la fermeture effective

Load, FormClosing, FormClosed : à quoi ça sert ?

- Load : permet un pré-traitement des données
 - Ouvrir un fichier, pré-calculer des valeurs, ...
 - *Remarque*: peut également être fait via le constructeur du formulaire (dans Form1.cs)
 - En ajoutant le code *après* le InitializeComponent() (auto-généré)
 - En fait plus logique, et préférable, de le faire via le constructeur
 - <http://stackoverflow.com/questions/3199504/how-should-the-form-load-event-be-used-compared-to-its-constructor-method>
- FormClosing : permet une dernière interaction avec l'utilisateur
 - Demander s'il veut sauvegarder ses données
 - Lui permettre d'annuler la fermeture
 - ...
- FormClosed : permet un post-traitement
 - Libérer des ressources, fermer un fichier, ...

Différence entre FormClosing et FormClosed

- FormClosing se produit juste *avant* la fermeture, à un moment où il est encore possible de l'annuler
- Le 2^{ème} argument qu'elle reçoit, de type FormClosingEventArgs, a deux propriétés utiles
 - **CloseReason** : indique *pourquoi* le formulaire est fermé
 - ApplicationExitCall : le code du programme a ordonné la fermeture
 - FormOwnerClosing : le formulaire "propriétaire" est en cours de fermeture
 - UserClosing: l'utilisateur a demandé la fermeture (ALT+F4, clic sur croix ...)
 - WindowsShutDown : arrêt système
 - **Cancel** : valeur booléenne indiquant si l'opération doit être annulée
 - Accessible en lecture et en écriture

```
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    e.Cancel = true;    // Annule la fermeture
}
```

Exercice

Ecrire une application qui

- Affiche (dans le formulaire principal) la liste des carrés des entiers de 1 à 20
- Lorsque l'utilisateur ferme l'application, demande confirmation et offre la possibilité d'annuler la fermeture
- Si l'utilisateur confirme, prononce « l'heure du décès » dans une MessageBox

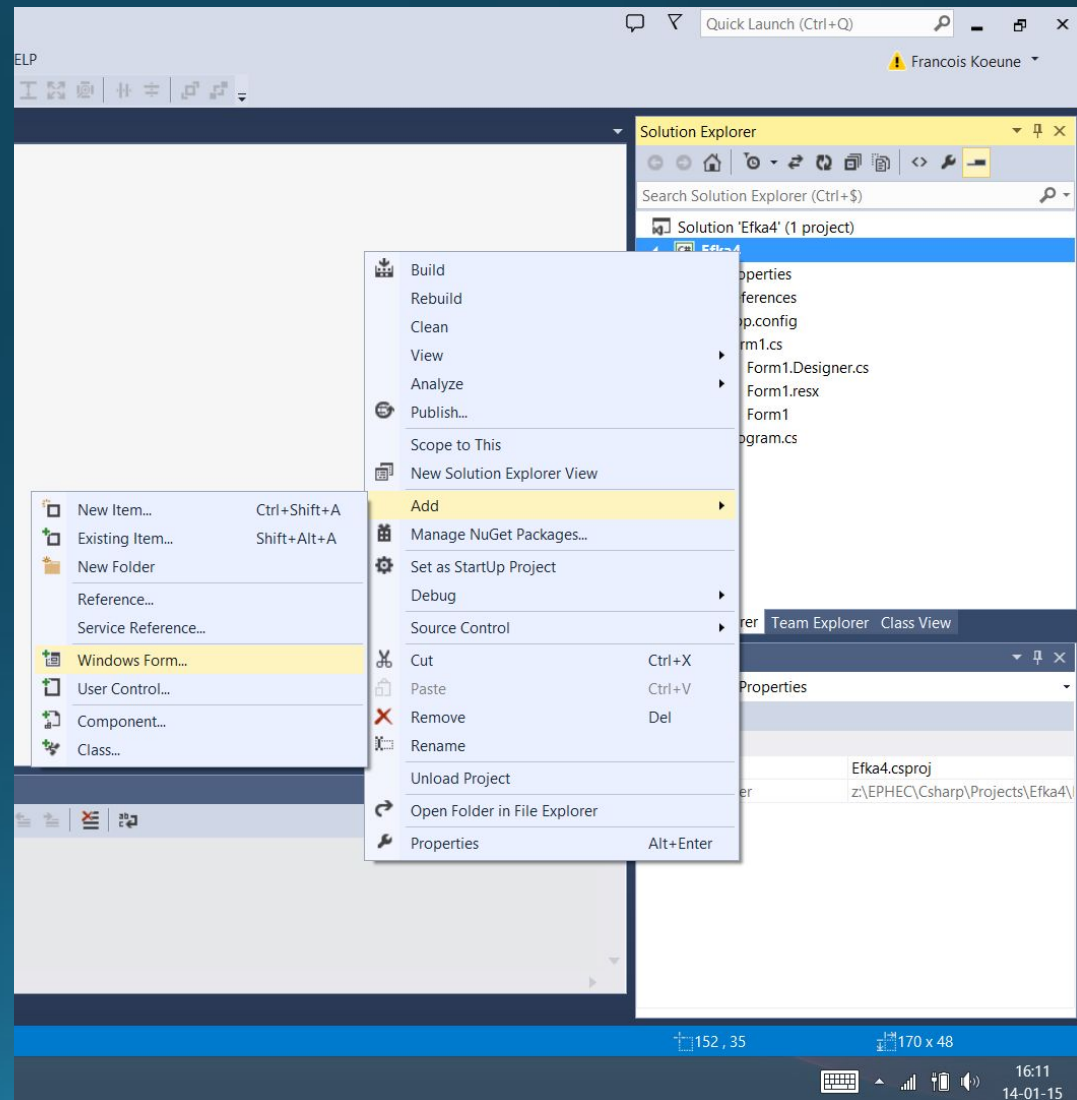
Second formulaire dans un projet

Second formulaire dans un projet

- Un projet peut comporter plusieurs formulaires
 - Un formulaire demande l'ouverture d'un autre,...
- Similaire à la classe MessageBox, mais
 - MessageBox est une classe statique, ce qui n'est pas obligatoire ici
 - MessageBox est limité à des boîtes de dialogue de format standard
- En pratique, il faut
 - Créer un nouveau formulaire (= une nouvelle classe)
 - Créer un objet de cette classe
 - Demander son affichage
 - Récupérer un résultat et/ou des attributs après sa fermeture

Création d'un second formulaire

- Dans le solution explorer
 - Clic-droit sur le projet
 - Add
 - Windows Form



Création d'un objet et affichage

- On crée l'objet formulaire comme n'importe quel autre objet
 - (ici, on a défini un formulaire (classe) nommé SaisieDonnées)
- On peut le faire afficher
 - En mode non-bloquant : méthode Show()
 - En mode bloquant : méthode ShowDialog()
 - L'utilisateur est obligé de traiter le second formulaire : le premier devient inaccessible tant que le second est affiché

```
SaisieDonnées F_Saisie = new SaisieDonnées();  
DialogResult Res = F_Saisie.ShowDialog();
```

Fermeture

- Comme pour tout formulaire, la fermeture peut être déclenchée
 - Soit par une action de l'utilisateur (<alt-F4>, clic sur croix...)
 - Soit par code: méthode Close()
- L'exécution reprend dans le code, juste après la ligne ayant appelé l'affichage

```
SaisieDonnées F_Saisie = new SaisieDonnées();  
DialogResult Res = F_Saisie.ShowDialog();  
// L'exécution reprendra ici après la fermeture du formulaire secondaire
```

Récupération de données

- Ce n'est pas parce que le formulaire est fermé que l'objet n'existe plus !
- Le code appelant a toujours accès aux attributs et méthodes de cet objet
 - Moyennant les limitations habituelles (uniquement public ...)
 - Remarque
 - Par défaut, les contrôles du formulaire sont déclarés private inaccessibles □
 - On pourrait les déclarer public
 - Ou, mieux, transférer les données en « pensant objet »

```
SaisieDonnées F_Saisie = new SaisieDonnées();  
DialogResult Res = F_Saisie.ShowDialog();  
// L'exécution reprendra ici après la fermeture du formulaire secondaire  
label_Output.Text = F_Saisie.TexteEntré;
```

```

public partial class SaisieDonnées : Form
{
    public string TexteEntré = ""; // Un attribut que j'ai défini "moi-même"

    private void button_Confirmer_Click(object sender, EventArgs e)
    {
        TexteEntré = textBox_Input.Text;
    }
    [...]
}

```

Formulaire secondaire

[...]

```

public partial class Form1 : Form
{
    private void button_Saisir_Click(object sender, EventArgs e)
    {
        SaisieDonnées F_Saisie = new SaisieDonnées();
        DialogResult Res = F_Saisie.ShowDialog();
        // L'exécution reprendra ici après la fermeture du formulaire secondaire
        label_Output.Text = F_Saisie.TexteEntré;
    }
}

```

Formulaire principal

Récupération de données : « penser programmation OO »

- Supposons que pour créer un objet `Personne`, on utilise un formulaire `SaisieDonnées` pour saisir les valeurs
- Ce formulaire comporte donc des `TextBox` « nom », « prénom », ...
- A la fermeture du formulaire, le code appelant peut récupérer les différents champs et créer l'objet `Personne`
- Mais cela ne respecte pas la philosophie OO !
 - Quel est le rôle de `SaisieDonnées` ?
 - Saisir les données relatives à une `Personne`
 - Dans ce cas, qu'il renvoie directement un objet `Personne` !
 - `SaisieDonnées` possède un attribut public `PersonneCréée`
 - Avant sa fermeture, crée l'objet avec les données saisies
- De manière générale, ne jamais hésiter à ajouter ses propres attributs/méthodes à une classe formulaire !

Renvoi d'une valeur

- Un formulaire peut directement renvoyer une valeur
 - Permet p.ex. de savoir ce que l'utilisateur a cliqué
- Propriété DialogResult (comme pour MessageBox)
 - On peut fixer cette valeur
 - Soit via l'interface Visual Studio, en fixant l'attribut DialogResult des différents contrôles du formulaire, par exemple
 - Bouton Confirmer ☐ DialogResult.OK
 - Bouton Annuler ☐ DialogResult.Cancel
 - Soit directement dans le code
this.DialogResult = DialogResult.OK
 - On la teste comme pour les MessageBox

```
DialogResult Res = F_Saisie.ShowDialog();  
if (Res == DialogResult.OK)  
{  
    label_Output.Text = F_Saisie.TexteEntré;  
}
```

Exercice

Ecrire une application comportant

- Un formulaire principal
 - Ce formulaire comporte juste
 - Un bouton "Créer personne"
 - Des labels permettant d'afficher les données de la personne créée
- Un formulaire secondaire permettant de saisir les données de la personne
 - Ce formulaire apparaît quand on clique sur le bouton "Créer personne"
 - Il comporte
 - Des Textbox permettant de taper le nom, prénom...
 - Un bouton "OK", un bouton "Cancel"
- Si l'utilisateur clique OK, le 2ème formulaire se ferme et le premier récupère les données **sous la forme d'un objet de classe Personne** et remplit ses labels
- Sinon, le 2ème formulaire se ferme, sans effet

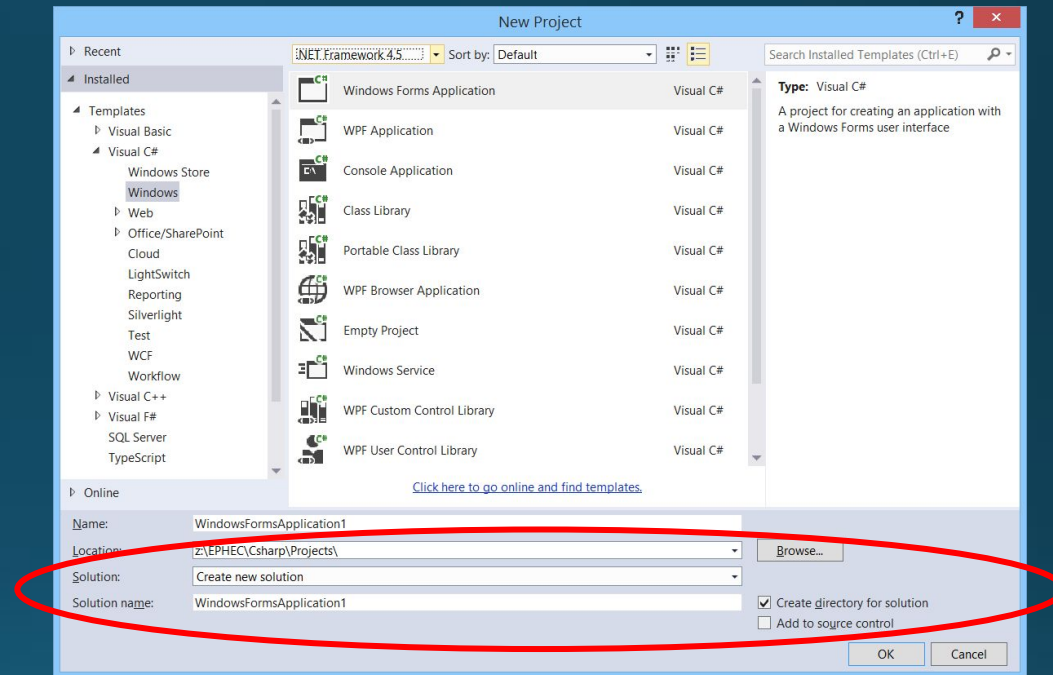
Partage de code entre applications

Partage de code entre applications

- Souvent intéressant de réutiliser des classes écrites pour une autre application
 - Supposons que l'on veuille programmer une interface graphique pour notre mini-jeu de rôles
 - Une bonne partie du code (attaquer, classe-fille Mage...) reste valide
 - On pourrait simplement copier-coller le code dans la nouvelle application
 - Mais redondant, 2 versions à maintenir en parallèle,...
- Préférable d'inclure le code d'origine dans la nouvelle application

Projets et solutions

- Chaque projet Visual Studio est inclus dans une solution
 - Créée par défaut à la création du projet
- Mais une solution peut contenir plusieurs projets

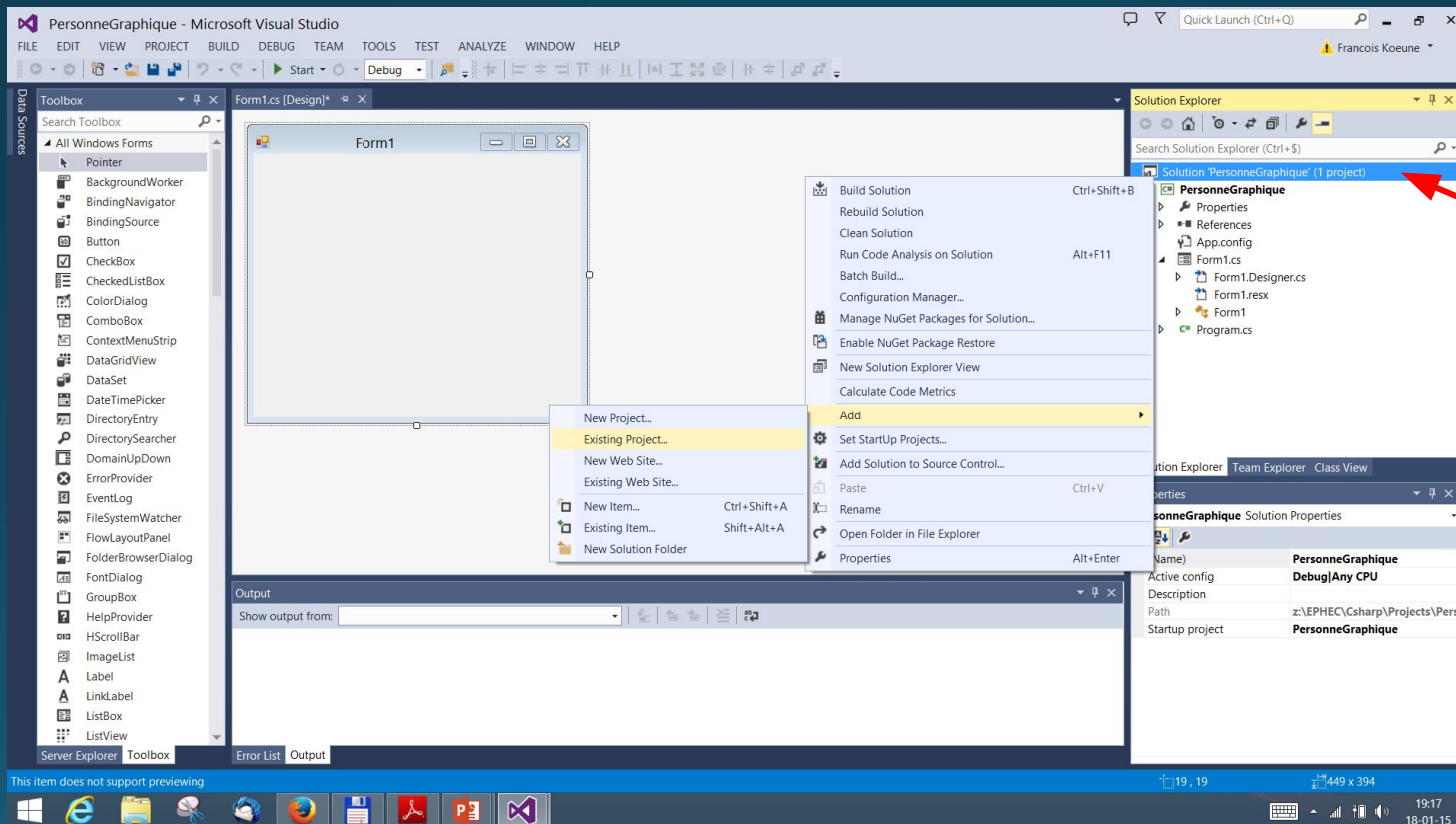


Partage de code : en pratique

- En 3 temps
 1. Ajouter le projet à partager à la solution
 2. Ajouter une/des références aux classes à réutiliser
 3. On peut alors déclarer des objets appartenant à une classe du projet partagé
 - En précisant leur namespace
 - Et à condition que la classe soit public

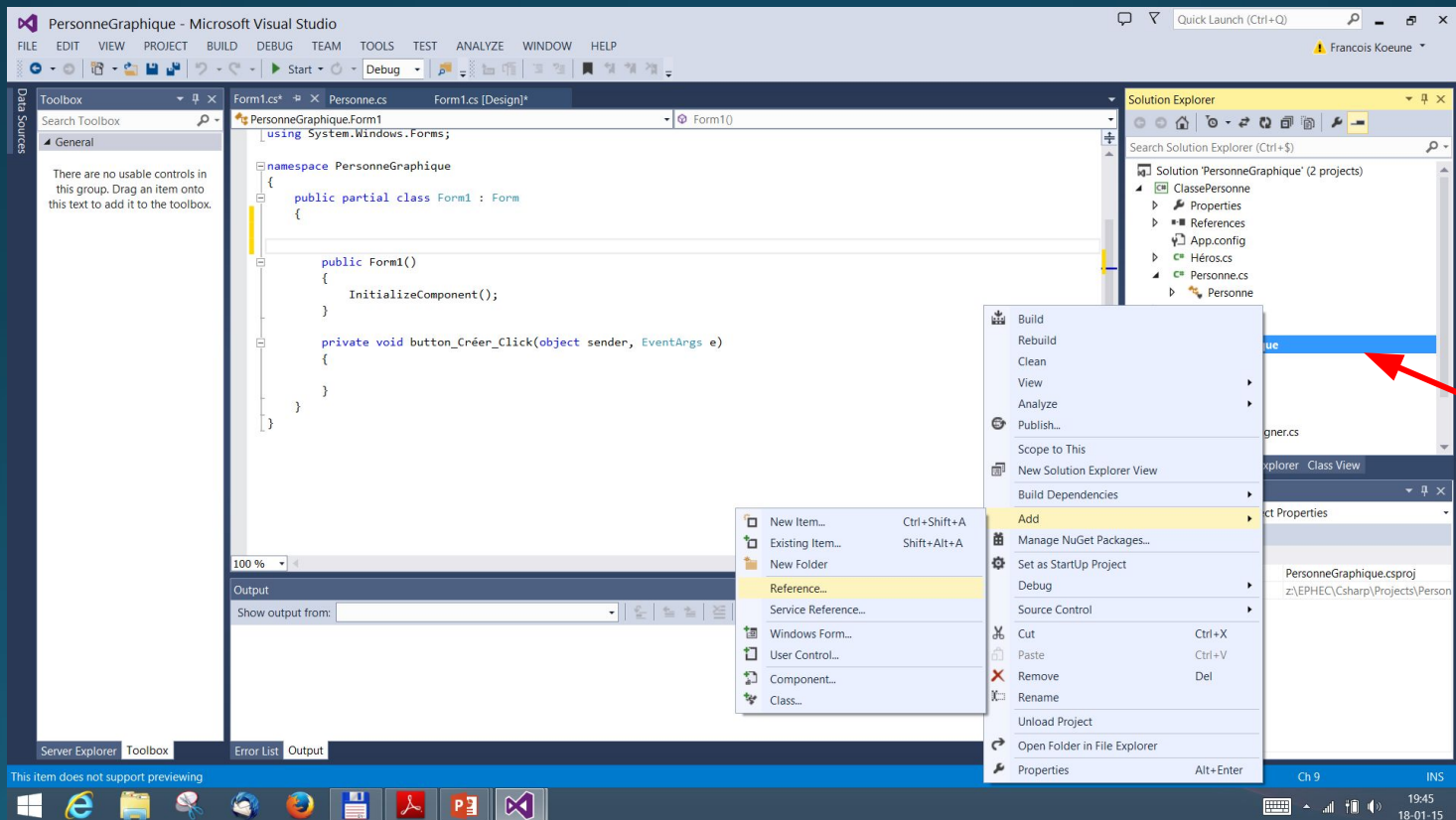
Ajouter un projet à la solution

- Solution Explorer □ clic-droit sur Solution □ Add □ Existing Project



Ajouter une/des références aux classes à réutiliser

- Pour que le projet A « voie » le projet B
 - Solution Explorer  clic-droit sur projet A  Add  Project Reference
 - Sélectionner le projet B



Utiliser la classe partagée

- On peut maintenant déclarer des objets de la classe partagée
- Mais attention, 2 pièges
 - Cette classe est dans un autre namespace, il faut donc le préciser
 - Soit en donnant le nom complet
 - Soit via la directive using
 - Seules les classes public sont accessibles de l'extérieur
 - Par défaut, une classe est private
 - Il faut donc la déclarer public

Dans la classe partagée

```
namespace ClassePersonne
{
    public class Personne
    {
        private String Nom;
        private String Prénom;
        private int AnnéeNaissance;
        [...]
    }
}
```

Dans le code qui la réutilise

```
using System.Threading.Tasks;
using System.Windows.Forms;
using ClassePersonne;

namespace PersonneGraphique {
    public partial class Form1 : Form {
        Personne p1; // Ou, sans using, ClassePersonne.Personne p1;
        [...]
    }
}
```

Remarque

- Il s'agit bien de *partage* de code, pas de copie
- Toutes les modifications apportées au code impactent aussi le code d'origine

Exercice

- Programmez une interface graphique pour votre mini-jeu de rôle
 - Voir le programme JeuDeRoleGraphique.zip sur le portail

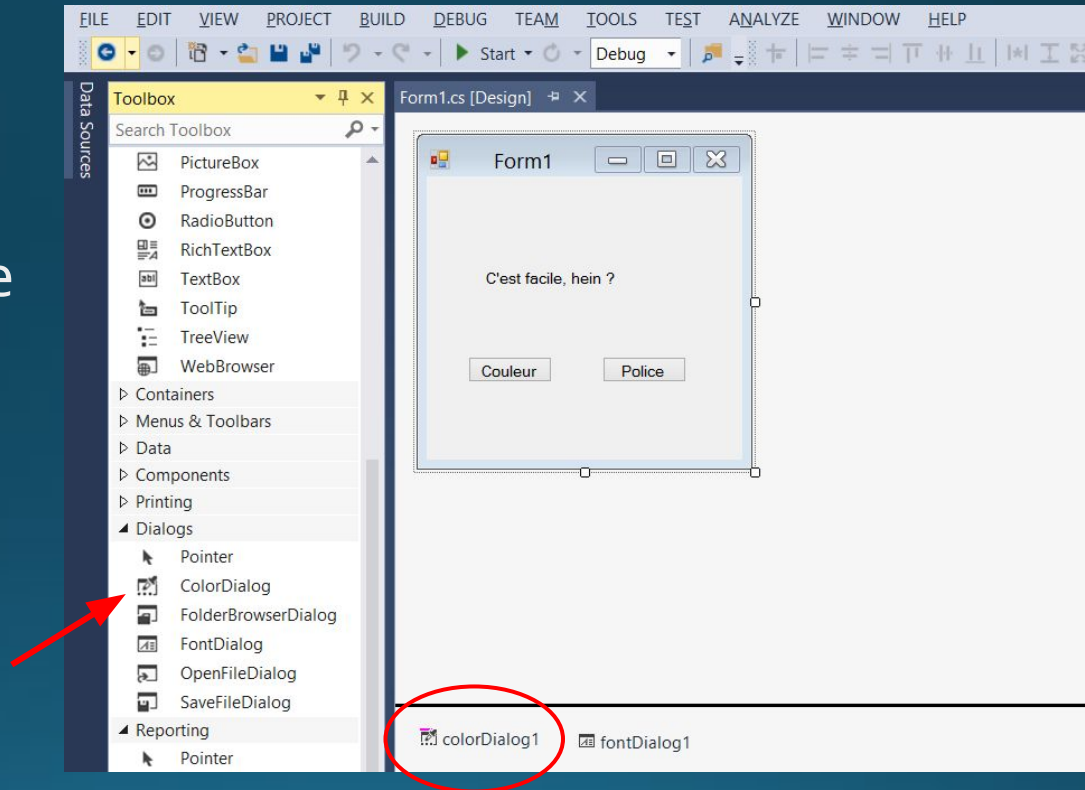
Composants non visuels

Composants non visuels

- Des composants qu'on ne voit pas, mais qui ont un effet
 - Timer : envoie des signaux à intervalles réguliers
 - OpenFileDialog, SaveFileDialog : pour choisir un fichier
 - **FontDialog : pour choisir une police**
 - **ColorDialog : pour choisir une couleur**
 - ...

Ajouter un composant non visuel

- On l'ajoute comme n'importe quel contrôle
- Il n'apparaît pas sur le formulaire, mais en-dessous



Utiliser ColorDialog et FontDialog

- Classes non statiques : il faut créer des objets
- Pour les afficher (p.ex. lorsqu'un bouton est cliqué)
 - Méthode ShowDialog
 - Renvoie un DialogResult indiquant le type de fermeture
 - DialogResult.OK, DialogResult.Cancel
- Après la fermeture, on peut récupérer le choix de l'utilisateur
 - Via l'attribut Color de l'objet de classe ColorDialog
- L'objet renvoyé **est** une couleur : peut être utilisé directement

```
label_TexteDemo.ForeColor = colorDialog1.Color;
```

- Idem pour FontDialog

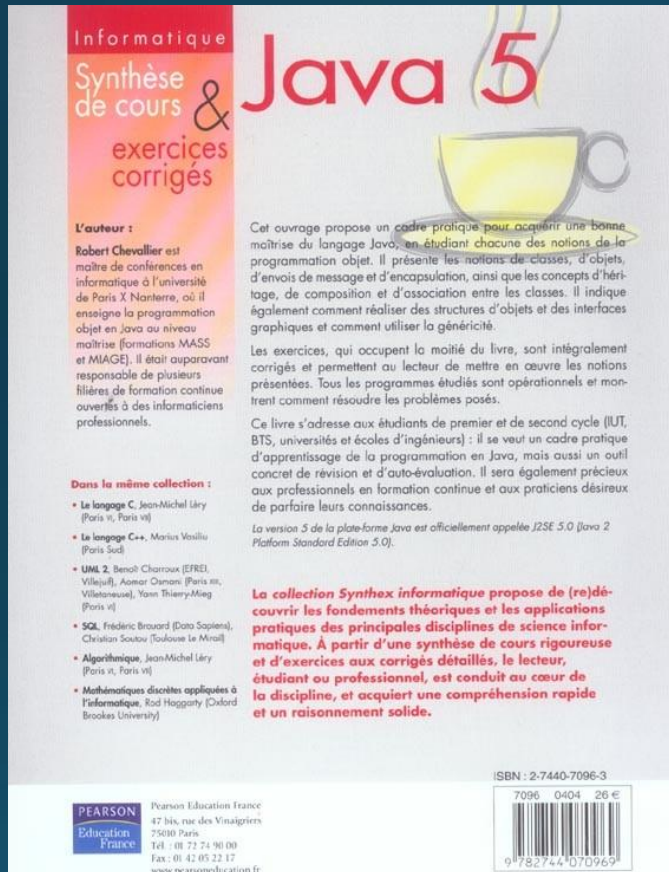
Exercice

- Créez une interface affichant un label et deux boutons, permettant de choisir la police et la couleur de texte du label
 - Attention : il faut que, si l'utilisateur clique « Cancel » pendant son choix, la couleur/police ne change pas

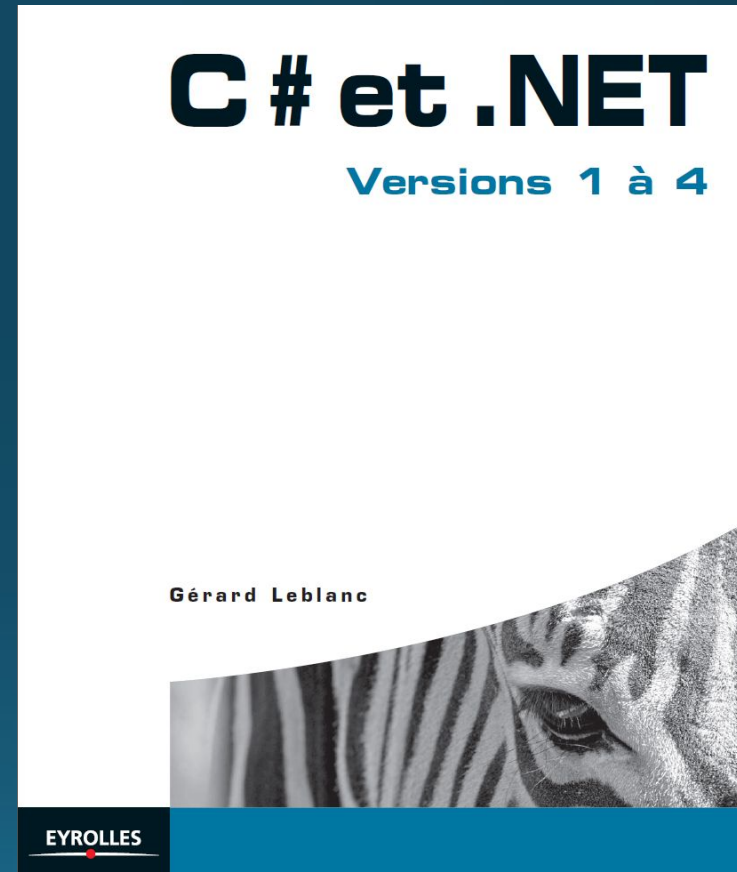
Exercice de synthèse

- Créez une application de gestion d'une bibliothèque
- Idées
 - Des **personnes** empruntent/rendent des **livres** et paient les emprunts et les amendes de retard
 - Un système de **messagerie** (email) permet à la **bibliothèque** d'envoyer des rappels en cas de retard
 - Une interface graphique permet de gérer les emprunts
- Seuls ou par groupes de 2
 - L'un écrit personne et messagerie, l'autre livre et bibliothèque
 - Prenez d'abord un moment pour décider ensemble des méthodes et attributs principaux (et de leurs noms)
 - Une fois terminé, voyez ensemble pour l'interface graphique

Références

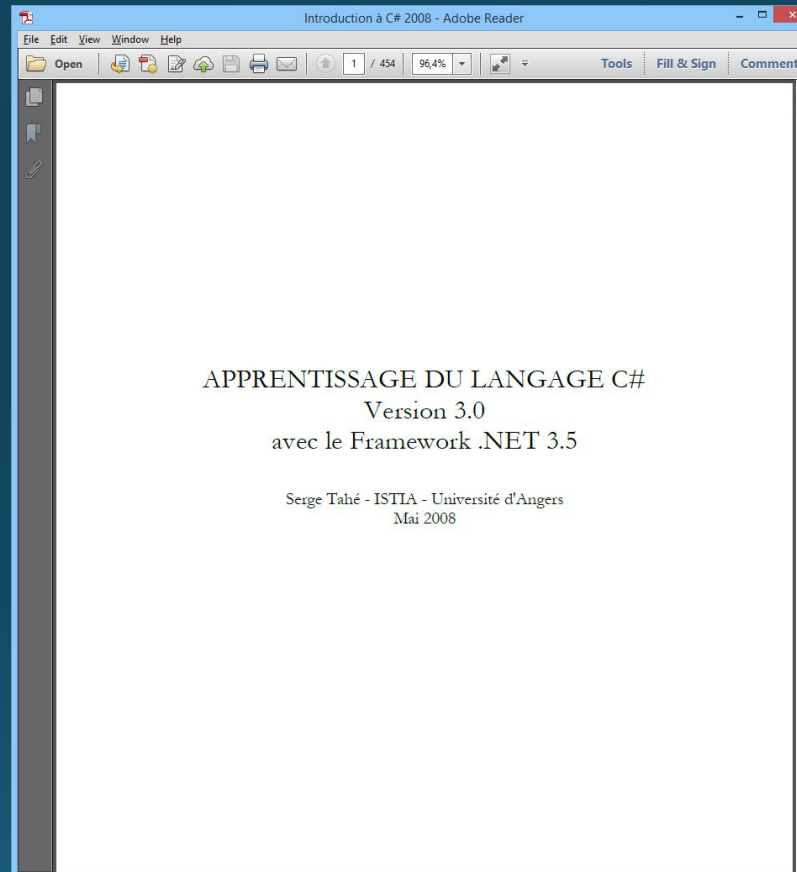


Java, mais bonne introduction de la philosophie OO



C#, mais manuel de référence plutôt que cours d'OO

Références



Disponible gratuitement sur
<http://tahe.developpez.com/dotnet/csharp/>
Surtout utilisé pour la partie interfaces graphiques

Quelques rappels concernant l'examen

Vous devez être capable de:

- Créer un projet du bon type
- Récupérer un projet que je fournis et le copier localement pour l'éditer
- Transférer un projet contenant vos réponses à l'endroit indiqué
- Ecrire du code syntaxiquement correct, et un projet qui compile

```

static string ChercherMot Try.Parse(string[] grille, string mot)

{
    int i = 0;
    string[] grille = new string[8, 9] { ligne, colonne };
    string mot;
    while (i < grille.Length)
    {
        i++;
        Console.WriteLine(grille);
    }
    List<string> Cherchemot = new List<string>;
    string c;
    Console.WriteLine( "Introduisez la premiere lettre");
    c = Console.ReadLine();
    chercherMot.Add('c');
    return c;
}

```

// Q10 chercher mots depuis la liste dans une grille.

```

static string ChercherMots(int[] Grille, string List<z>)
{
}

```

- La classe `ConsoleColor` permet de définir des couleurs liées à la Console
 - On peut s'en servir pour accéder à des constantes, p.ex.
`ConsoleColor.Green` = "la couleur verte"
- La classe `Console` possède un attribut `ForegroundColor`, de type `ConsoleColor`